Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Search and Rescue Quadruped

(SaRQ)

Project Engineering

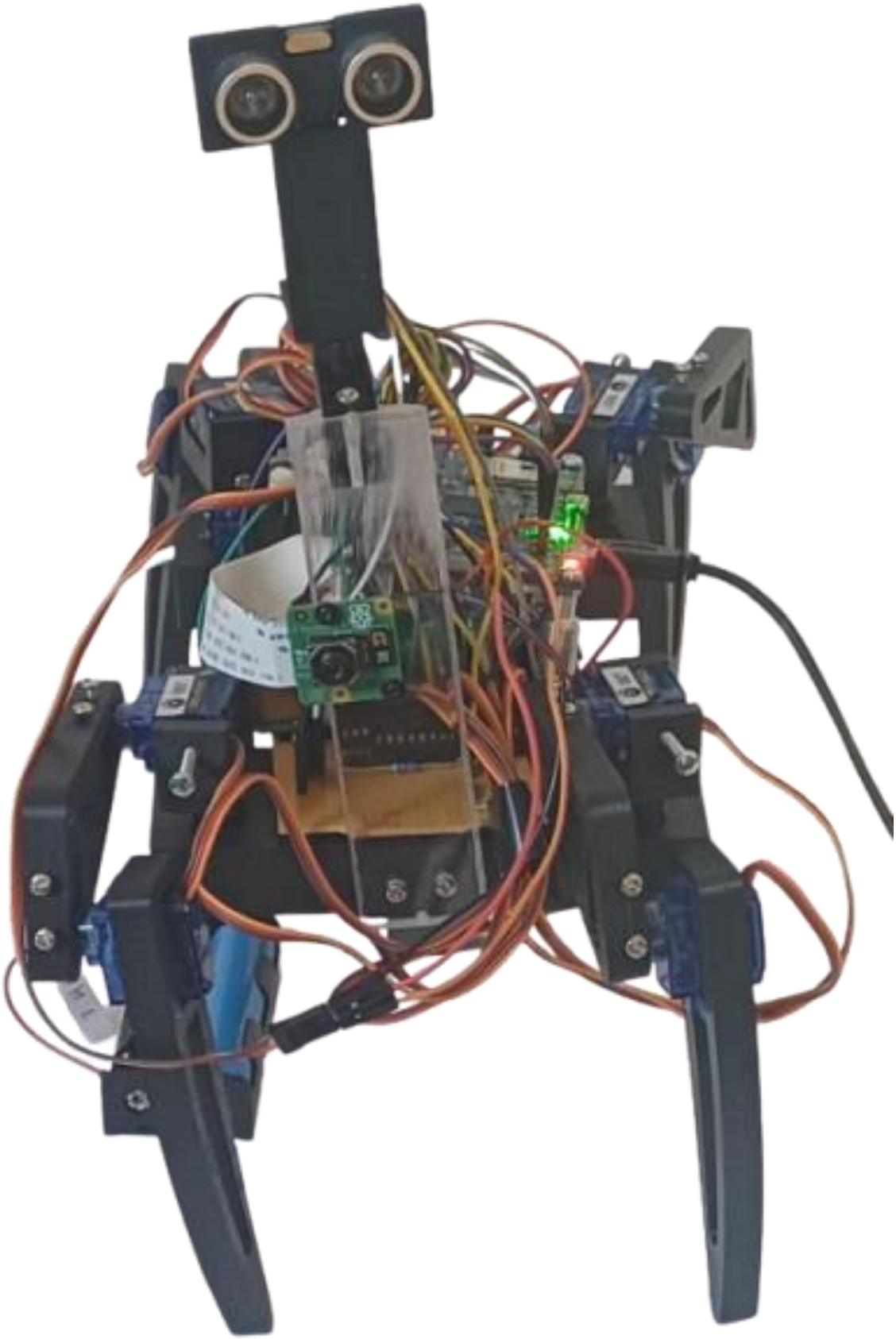
Year 4

Nathan Ferry

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026



Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

_____Nathan Ferry_____

Acknowledgements

I would like to thank my project supervisor, Niall O Keeffe, for his assistance and guidance throughout the duration of my final year project. I would also like to thank all my fellow classmates that allowed me to throw ideas at them and aided me throughout.

Table of Contents

1 Summary

The goal in creating the search and rescue quadruped (SaRQ) was to design and build a robot to deploy in disaster scenarios. With the SaRQ, injured persons could be located within hazardous or unstable buildings, and personnel alerted.

The SaRQ's scope included a range of goals such as getting the SaRQ to walk, detecting humans, designing the legs and chassis of the robot, implementing inverse kinematics (IK) and alerting the search and rescue personnel (SARP) that a human had been detected.

The first feature implemented was universal asynchronous receiver-transmitter protocol (UART), for serial communication. This was between a Raspberry Pi 4 model B (RPI) and an STM32L475VGT6 (STM32) microcontroller unit (MCU) on the STM32IOT1A. The RPI as the logical centre of the robot, informs the STM32 of the next movement while communicating with devices such as a Pi camera V3 (Pi cam) and HC-SR04 ultrasonic sensor. The HC-SR04 prevents collisions, while the Pi cam uses the *you only look once* (YOLO) algorithm to detect people. If a person is detected, the RPI runs a simple mail transfer protocol (SMTP) server, sending an email to SARP with images attached. A live feed of the Pi cam's view is streamed via a motion JPEG (MJPEG) server. The STM32 controls the legs of the SaRQ via a series of servo motors. The legs and chassis were drawn, printed and tested rigorously. IK mapped the movement of the SaRQ in a co-ordinate system, acting as a tool in creating walking animations.

The RPI was programmed in Python over secure shell in the Visual Studio Code integrated development environment (IDE). The STM32 was programmed in C in the STM32CubeIDE. The project followed agile methodologies, and Jira was used for planning sprints and keeping an up-to-date backlog. OneNote for documentation in the form of weekly logs, standups and detail on specific sections such as leg design. All 3D prints were drawn in Onshape.

In completing this project, I have a deeper understanding of integrating hardware and software in a cohesive project. My knowledge of machine learning has deepened, along with SMTP protocols and serial communication. My design skills and problem solving improved when creating the design of the SaRQ and adding IK to the project respectively. In terms of soft skills, I enhanced my documentation skills and planned sprints more appropriately.

2 Poster

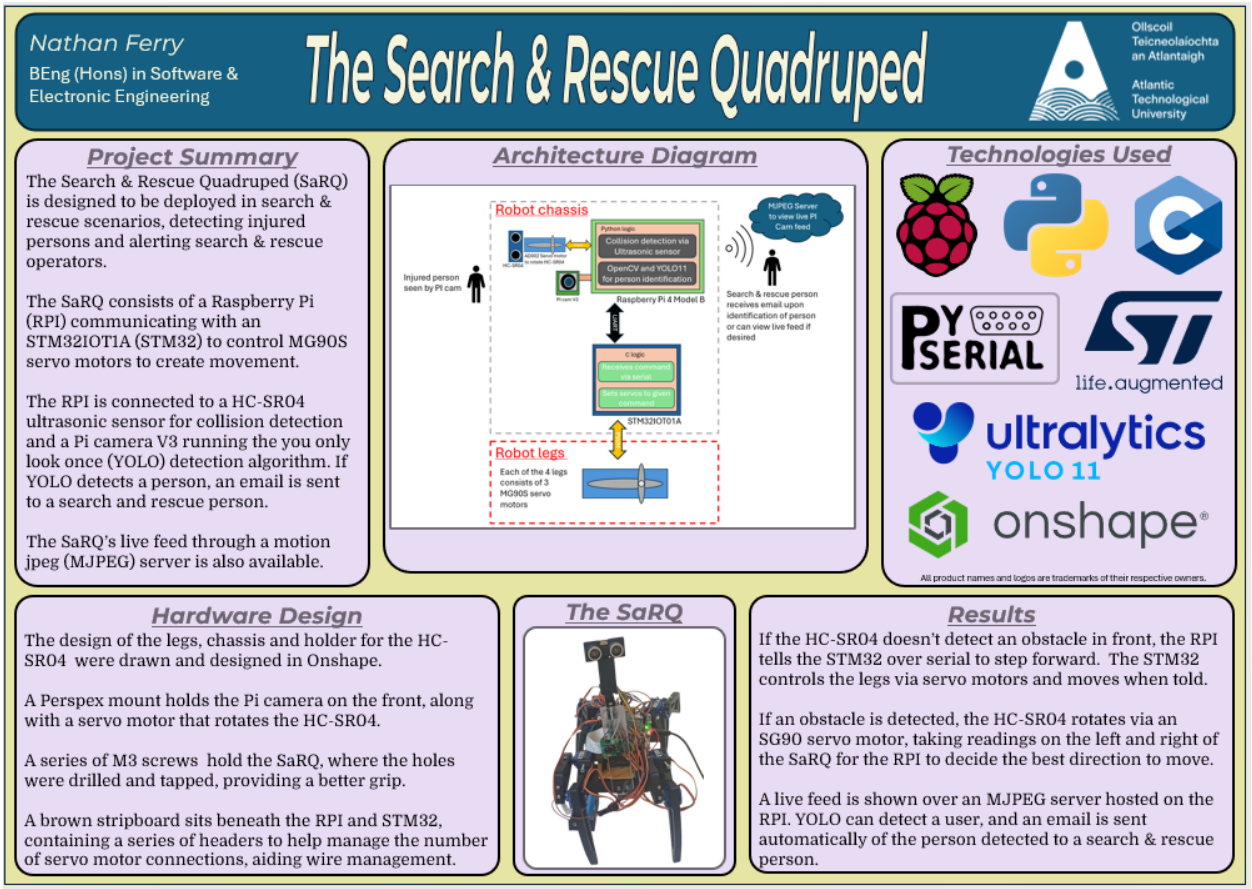


Figure 2.1 SaRQ Poster

3 Project Architecture

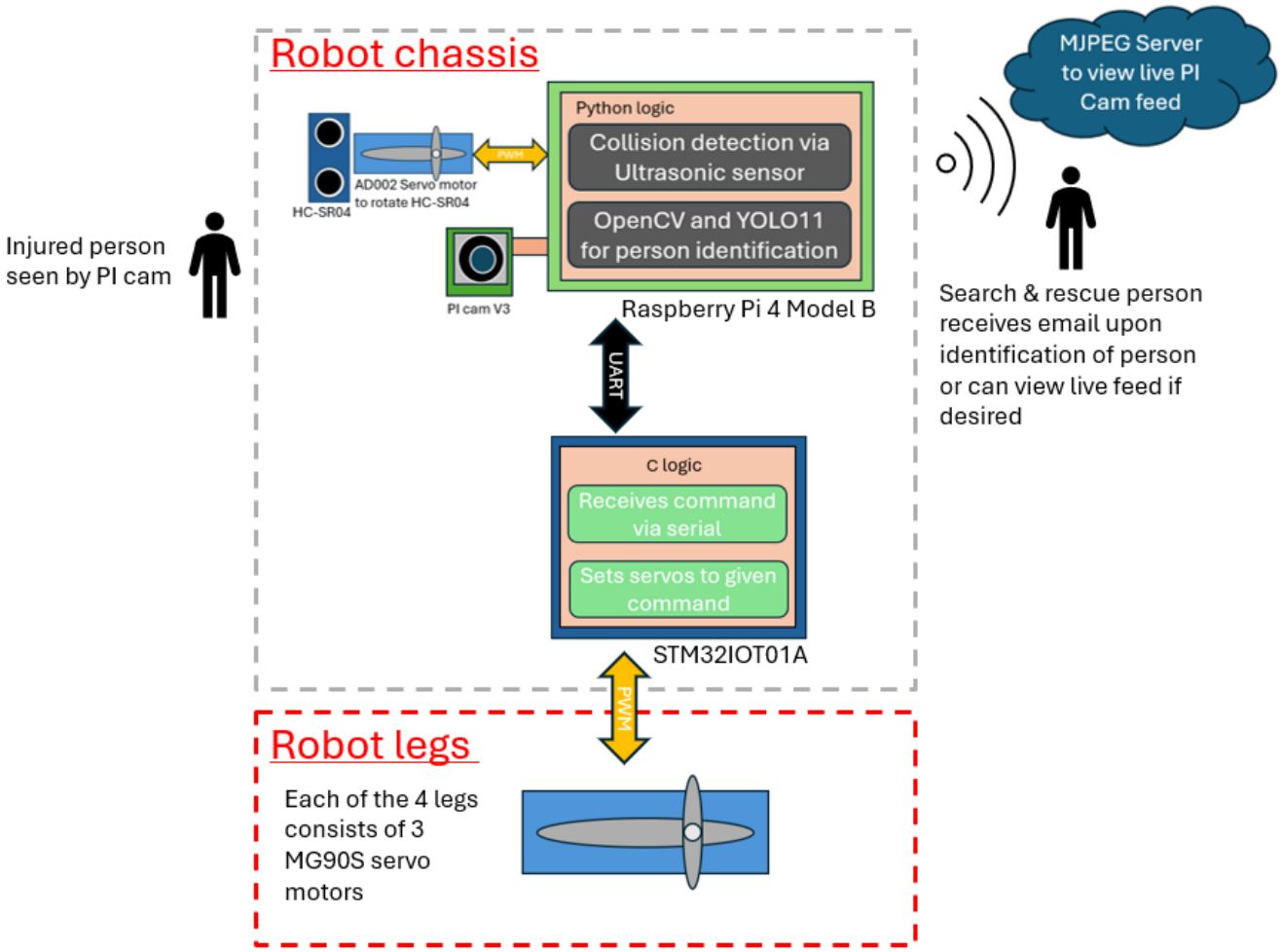


Figure 3.1 SaRQ architecture diagram

4 Introduction

Natural disasters are unpredictable for SARP risking their own safety to save the most lives. During earthquakes, they search rubble at risk of collapse on the hopes that an injured party may be there. For my project, I designed and built the SaRQ to aid in disaster situations. The SaRQ could be sent in the place of a rescue operative to search for potential persons in need of aid. It would locate an injured person and notify SARP. This robot could be deployed in the aftermath of earthquakes to help alleviate the risk on search and rescue teams, helping people have a better chance of being found. I care about people and want to use the knowledge gained from my course on the best possible task I can.

The project had a wide scope, challenging me to learn about topics I was unfamiliar with such as implementing YOLO for detecting people with the Pi cam, or using an SMTP server for sending emails. This was contrasted with hardware I was familiar with, such as the HC-SR04, RPI and STM32. I understood UART before this project and had designed components for 3D printing in Onshape previously, but this project gave me the opportunity to implement UART to give me a deeper understanding of the protocol and design the SaRQ from the ground up, letting my Onshape design come to the fore. I had an interest in mathematics but not used it when programming until now. When adding IK to the SaRQ, I understood how robots moved in the real world and saw the complexities involved. I had not heard of MJPEG until I added it to this project but felt that the SaRQ benefitted immensely form having a secondary form of detecting persons. If email attachments were blurry, there would be no backup otherwise, but with the MJPEG server, SARP can check the SaRQ's view. The servo motors used were programmed with pulse width modulation or PWM (see the timers section for more detail) and challenged me in understanding the theory of operation. More importantly, it pushed me in how I would get the SaRQ to balance. Walking was trial and error, attempting to get the balance right and repeated motions correct. Building the SaRQ to be sturdy was one of the most important parts of the project but taught me the most about robotics in the real world. Gravity is messy and counteracting forces can be tricky, but in the end I persevered.

This report is separated into three core sections for the project. The first focuses on the STM32 and implementing serial communication and timers, the second section follows the RPI and creating scripts such as an inverse kinematics file and a main program which uses YOLO, an MJPEG server and an SMTP server. Finally, the report will conclude on the mechanical side of the project, focusing on designing and assembling the SaRQ mechanically.

5 Part I: The STM32 in the SaRQ

5.1 The STM32L475VGT6

The STM32IOT1A is a microcontroller with a powerful ARM Ultra-low-power Arm Cortex-M4 32-bit MCU. The microcontroller comes with an Arduino form-factor and a series of useful peripherals and sensors such as HTS221 for reading temperature and humidity, VL53LOX time of flight controller for gesture control among many more [3][4]. This project uses the microcontroller as a central piece, but the peripherals are not of interest in this project, the STM32L475VGT6 MCU is. The MCU has 1 megabyte of flash memory to store programs and data, a power supply in the range of 1.71 volts to 3.6 volts, a frequency up to 80 megahertz and multiple clock sources from a 4 to 48-megahertz crystal oscillator to 32 kilohertz crystal oscillator for real time clocking. There are up to 128 kilobytes of static random-access memory (SRAM). There are 114 input/output (I/O) pins [3]. Each of the general-purpose input/output (GPIO) pins can be configured via software to be an output, input or as a peripheral alternate function [3]. Most GPIOs are 5V tolerant [3]. There are 16 timers, 2 16-bit advanced motor control timers, 2 32-bit timers, 5 16-bit general purpose timers among many more [3]. There are also 20 communication interfaces, including USB, 5 Universal Synchronous/Asynchronous Receiver-Transmitters (USART), 3 serial peripheral interfaces, 1 low power UART, 3 inter-integrated circuits (I2Cs) among many more. Overall, the STM32L475VGT6 has a great deal of uses, but the main case for my project was in general purpose timers and using USART. The SaRQ has a series of servo motors integrated into 3D printed components to form legs. The STM32L475VGT6 has one main objective, control the SaRQ's servo motors. Further on in this section, you will learn the theory of operation behind how this occurs, as well as how I programmed this MCU in the embedded c language. The STM32 is a part of the larger architecture of the SaRQ, whereby it communicates over UART with the RPI or the brain of the SaRQ . The RPI decides in which direction the SaRQ should move and sends this decision to the STM32 which in turn sets the servo motors accordingly.

5.2 The Servo Motors

I used two servo motor models over the course of this project with the same theory of operation. The servo motor model used in the legs of the SaRQ is the MG90S which has metal gears and are hence stronger than plastic gears with a ground, power and signal connection (see figure 5.1) [1]. The AD002 is used for rotating the HC-SR04 for collision detection and operates in the same voltage range with the same pinout for the MG90S [1][2]. They operate with a 20 millisecond (ms) period in the at 4.8 volts [1][2]. I prompted ChatGPT when initially researching servo motors for models it would recommend given my project scope and the MG90S was its choice.

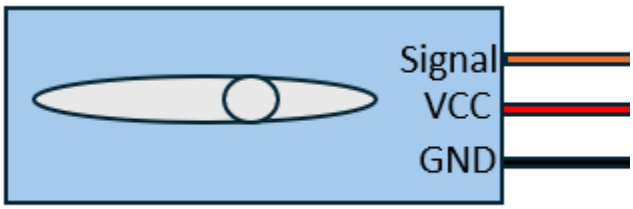


Figure 5.1 Servo motor pinout

5.3 Timers

Timers were an enormous part of this project, allowing me to use PWM to programme the servo motors used for the SaRQ's motion. I used an STM32 to control the servo motors due to my previous experience with the microcontroller the STM32 sits on, and the number of timers and pins the board had available to use. The goal was to create a function to handle all underlying logic, where I would pass an angle desired, and the function would handle the rest. Before handling that, I needed to understand what was going on underneath it all, at the register level.

5.3.1 What is PWM?

To put it simply, PWM is a means of controlling a servo motor's angle based on the current you feed it. This current comes in the form of a periodic square wave. The longer the square wave is high, the greater the current and the more the servo moves. For example, if the wave is causing the servo to be at 10 degrees and the time high is lengthened, the servo will move further (i.e.

20 degrees, 30 degrees and so on). Note that the wave being the same width means that a specific angle will be held. These servo motors require the frequency of this wave to be 50Hz and hence the period of the waveform sent to the servo motor would be 20 ms as frequency is inversely proportional to time (Frequency (Hz) = 1/Time (s)) [1].

5.3.2 PWM on the STM32

You may be wondering, how does this work on the STM32? To put it plainly, it is all about registers. Registers dictate how long the wave stays high, the resolution of the motion and in the end, are the main hurdle around moving servo motors with the STM32. If you understand what the registers are doing, you understand how to program servo motors on this hardware. I've spoken about how PWM works in an overview, but how does the STM32 set the width of the pulse sent to each servo motor?

$F_{Hz} = F_{clk} / (ARR + 1)(Prescaler + 1)$

The equation above explains how you can set a timer for a specific frequency and use register values to get the highest resolution attainable for each timer. F_{Hz} is the frequency of the wave in hertz. In this instance, we are looking for 50Hz for the servo motor. F_{CLK} is the clock frequency. The STM32 was set up with an 80MHz frequency. Due to the STM32 having an 80MHz frequency, it will take a certain time to count each number. The auto reload register or ARR is the max count of the timer. If you want a 50Hz wave, this number and the prescaler need to divide the clock frequency by enough so that it takes 20ms to count to the ARR. The prescaler is used to divide the clock frequency and should be kept as low as possible while the ARR should be as high as possible. The ARR dictates the max number the timer will count to. This is useful as the higher this number is, the higher the resolution of motion (see figure 5.2).

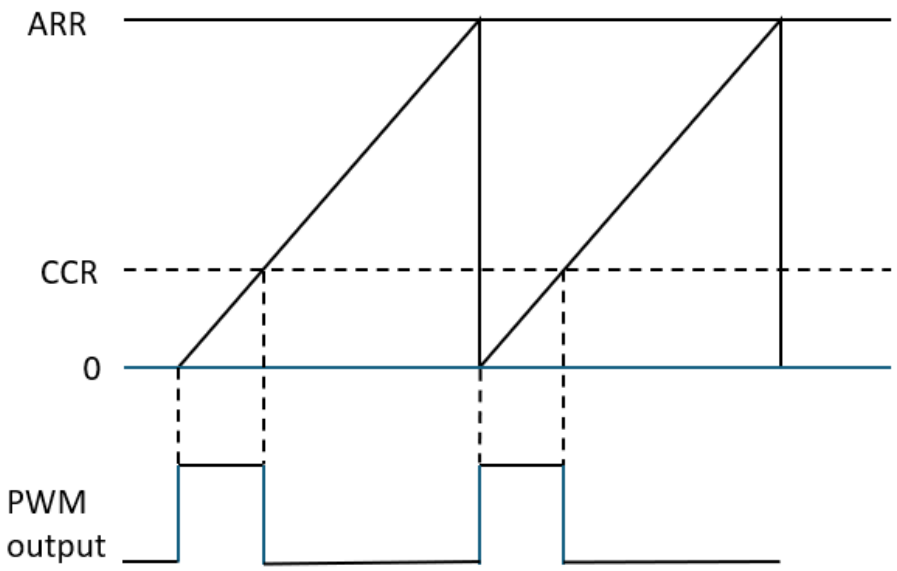


Figure 5.2 CCR resulting in PWM output

There is one other register that has been omitted from my explanation until now. The ARR, prescaler and above are all the same for each servo, but the compare/capture registers or CCRs are what dictates individual servo motor settings. Each timer has several channels, and CCRs associated with those channels. The diagram above shows a series of triangle waves with a square wave underneath. The triangular waves are the counts to the ARR. The count is linear and once complete, the count is reset and begins again. The dashed lines show that based on the CCR the square wave is generated. The ARR will still count, but the CCR value is the cutoff for the square wave. By making the CCR larger, the wave will be high for longer. This is why having the ARR be as high as possible increases the resolution. If the ARR is 10 then the CCR can't control the width of the wave as much as if the ARR was 100. In the first example the width of the wave differs by 10% but in the second it could change by 1%. Each timer channel has an individual CCR. An example would be channel 1 of timer 16. The channel is used to control a servo. CCR1 of timer 16 would be set to a value which would generate a wave to move the servo a specific angle. I chose the same ARR and prescaler for each clock which ended up being 63999 and 24 respectively. Note, the ARR starts from 0 and hence the count is 64000.

5.3.3 Mapping the Pins

I programmed in the STM32CubeIDE where the project consisted of a file where my main code resided (named UserApp.c), and an .ioc file where the timers were enabled and set. Note, there were many headers and other files, but they are not relevant as the majority of the programming was done in those two files. The SaRQ has three servos per leg for a total of 12. I had to map 12 servos to designated timers in the .ioc of my project. This was where the documentation was required. I used the datasheet for the MCU to find tables of the alternative functions for each pin [3]. I created a list of timers available and tried to map 12. The difficulty in this was the fact that some timers would overlap. For example, the pin PB4 had timer 3 channel one available, but it was in use by PA6. It was tricky to get everything connected, but in the end, I created a pinout map for the microcontroller's pins featuring timers and UART connections (serial communication is covered further on in the report).

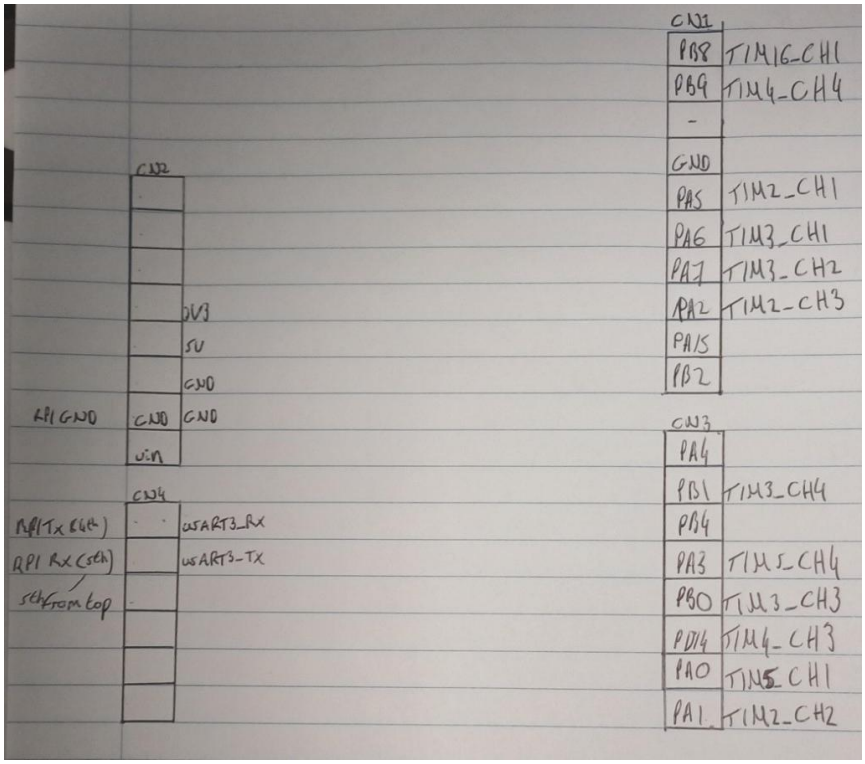


Figure 5.3 Pinout map

With the pins mapped, the .ioc setup was straightforward, consisting of selecting a pin and enabling the timer on it. On the left, I had to select “Timers” in the “Pinout & Configuration” panel before selecting each timer and selecting the channel under the “Mode” window. On a channel, I had to click the dropdown and set it to be “PWM Generation CHX” (X being a placeholder for the channel number in this instance). In the “Configuration” window, I input the relevant values for the prescaler and ARR. The final step to configure the timers was to hit the gear icon or “Device Configuration Tool Code Generation” to generate the code for the timers.

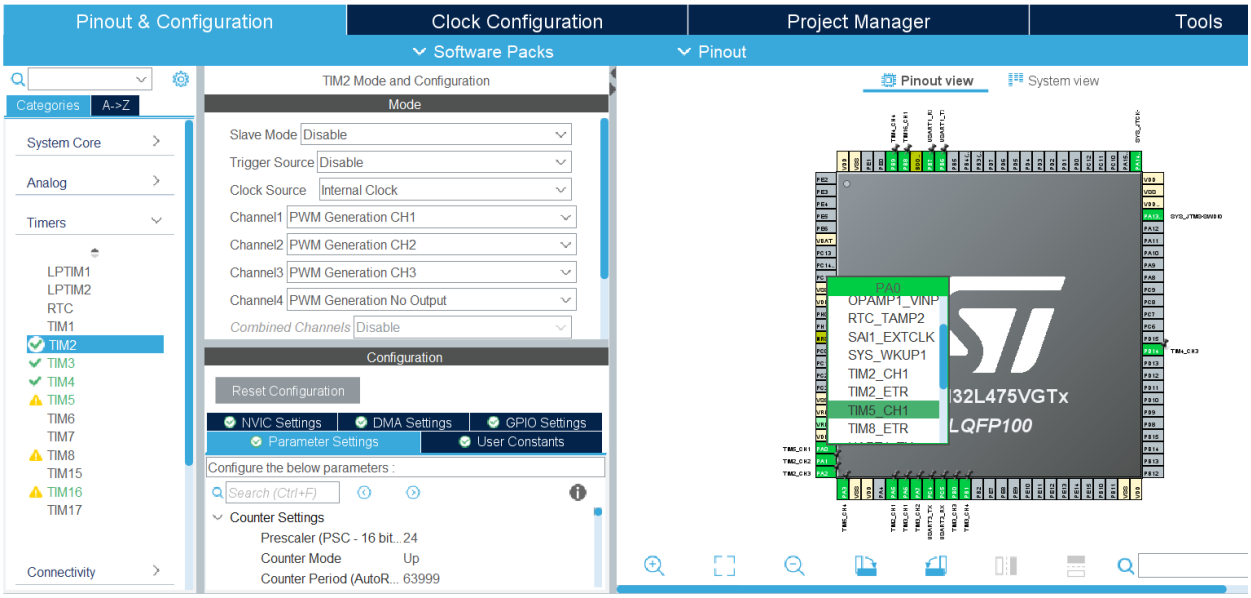


Figure 5.4 Timer configuration panel (left) & pin setting (Right)

5.3.4 The Timer Function

The final code starts so that all the timer registers are set to 0 and PWM is started per channel before entering the while(1) loop consisting of the main project code (see figure 5.5).

```
//TIM 16
TIM16->CCR1 = 0;
HAL_TIM_PWM_Start(&htim16, TIM_CHANNEL_1);
```

Figure 5.5 Initialising PWM and setting registers for timer 16

With the setup complete, a function was made to control servo operation. I tested a servo motor to see if the range of motion was 180 degrees as expected. The average range for PWM

of servo motors moving in 180 degrees is a high pulse anywhere between 0.5ms and 3ms of the 20ms period. I started by taking a single servo motor and testing the range of motion based upon differing CCR values. The way to calculate the CCR for 3ms would be to take the ARR (which would equate to 20ms) and divide it by 20 and multiply by 3. Trying out different ranges led me to the discovery that my servo motor would work in the range of 0 – 220 degrees. This was expected as the servo motors I have are cheap, but I didn't want to use the range of motion fully in case there were inaccuracies and unforeseen consequences to using the servos at their max range. I made a scale and tested the ranges of all the other servos to make sure they were the same. I had to tweak the range for the scaler to be between 0 – 210 to compensate for servo motors that would not go the full range. With this complete, I made a servo function that would accept three arguments: the angle, the channel and the timer. I had the CCR ranges for 0 and 210 degrees and used an equation to calculate the angle set in terms of a CCR value (see figure 5.6).

```
void servo(float anglePassed, uint8_t channel, TIM_HandleTypeDef *htim) {
    //Servo function calculated CCR value and returns it, angle is 210 as it is the servos max.
    float CCR_Return = 0;
    uint16_t Min_CCR = 1120, Max_CCR = 8495, AngleRange = 210;
    CCR_Return = ((Max_CCR - Min_CCR) * anglePassed) / AngleRange + Min_CCR;
    CCR_Return = roundf(CCR_Return);
    if (htim->Instance == TIM2) {
        switch (channel) {
        case 1:
            TIM2->CCR1 = CCR_Return;
            break;
        case 2:
            TIM2->CCR2 = CCR_Return;
            break;
        case 3:
            TIM2->CCR3 = CCR_Return;
            break;
        default:
            break;
        }
    }
```

Figure 5.6 Part of the servo function

The equation takes the difference between the max CCR and the minimum and multiplies it by the desired angle. This is divided by the angle range and the minimum is added. The minimum is added in case the value passed is 0. The maximum has the minimum removed from it to compensate for the fact that the minimum CCR is added at the end. The angle is divided by the

range so that the CCR value isn't greater than 180 degrees. Since the maximum range of motion of the servo motor is 210 degrees, the passed angle is simply divided by that range. This means the CCR is technically in the scale of 0 to 1 of the 210 degrees. After the CCR is calculated the result is rounded as an angle can be passed as a float but the registers cannot take non whole numbers. A series of if statements are used to find the timer. The channel is found via case statement and the CCR is set to the calculated value to move the servo motor. The function is larger than the image above, consisting of five if statements. The function is large, but it saves space in the walking logic where the servos are called (see figure 5.7).

```
servo(50, CCReg1, &htim16);
```

Figure 5.7 Calling the servo function

5.4 Serial Communication

UART is a protocol I implemented to communicate between the RPI and STM32. The RPI tells the STM32 how to program the servo motors. I had to use UART to send information between both devices so that the SaRQ avoids stepping into a collision. The RPI has external devices connected to sense humans and collisions (further detail of this is found later in the report). There is a second instance of UART communication within the project, the serial wire from the microcontroller to the device I program which displays print statements via a Putty terminal. Any time I print messages to the terminal, UART communication is used. This first case was already implemented in the project base, and I implemented the communication between the RPI and STM32 myself.

5.4.1 What is the UART Protocol?

Before delving into what UART is, we must understand why it was chosen. The UART protocol was chosen over others such as the serial peripheral interface (SPI) or I2C due to my familiarity with UART on the microcontroller. I had performed communication with UART several times in labs, and I had knowledge of the functions that are used for receiving and transmitting messages over UART via the STM32. It should be noted that SPI requires four wires whereas UART and I2C need only two, which was a reason to discount SPI [5][6]. UART can only handle one master and slave at once, but in this instance, it suited the project requirement [7]. If I

needed multiple slaves for the RPI, I would have chosen I2C as it supports that while requiring less connections than SPI [5][6]. Note, a master is a controlling device, and a slave is a listening device. In my project the RPI is the master as it controls the STM32.

The UART protocol uses two wires between a pair of devices. One wire transmits from a device and is the receiver on the other device (hence the RT in UART). The opposite is true on the other wire (see the orange wires in figure 5.8).

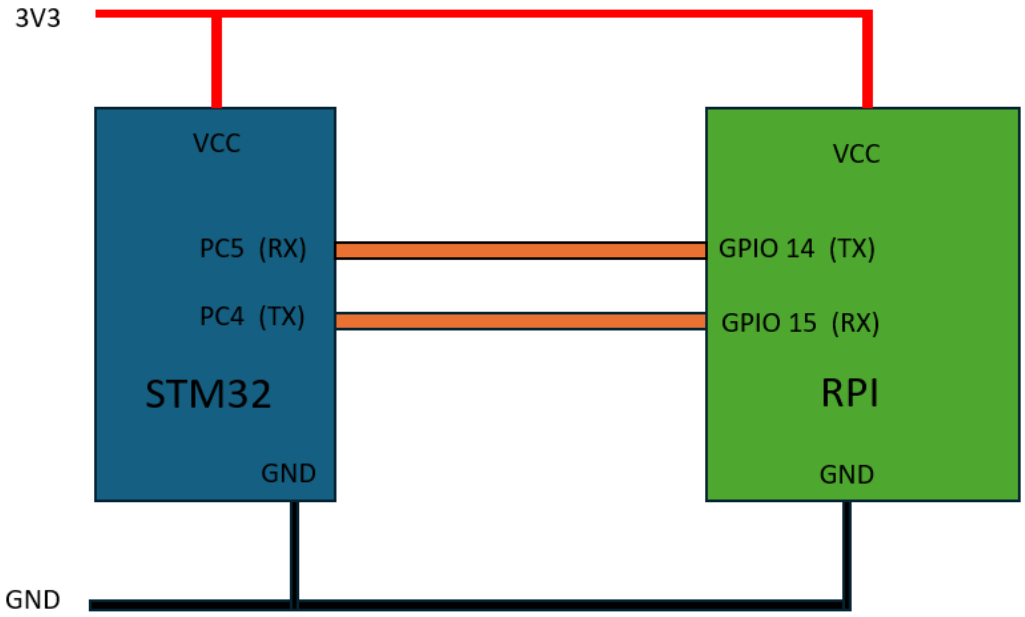


Figure 5.8 The UART communication between the RPI and STM32

UART is asynchronous, meaning there is no clock to synchronise the output bits from the transmitting side to the receiving side [7]. Data is transmitted in packets from device to device. There can be start bits, stop bits and parity bits [7]. The data transmission line is held high when data is not transmitted and the start bit is used to pull this voltage down to signal the beginning of data transmission [7]. The middle portion of the packet is simply data and can be in the range of 5 to 9 bits [7]. The parity bit is optional and used to check if the data has changed over transmission due to electromagnetic radiation, baud rates (the set speed of bits per second transferred) etc [7]. The parity bit can be set to even or odd parity, counting the number of 1s. If in even parity, the bit will be set high if there is an even number and low otherwise [7]. Odd

parity is the converse [7]. The stop bit or bits are used to signal the end of transmission by setting the voltage high for one or two bits at the end of a transmission [7].

5.4.2 UART on the STM32

The implementation of UART was straightforward as I had previous labs to reference for functions as well as the documentation I used when mapping the timers. I began by searching for UART compatible pins that would not conflict with my timer pins. Luckily, I found out that PC4 and PC5 were unused for timers and had USART3 as an alternate function available. While it is called USART3, I used it in UART mode. I went into the .ioc file and left clicked on each pin selecting “USART3_TX” and “USART3_RX” respectively. Unlike for the timers, I selected “Connectivity” on the left-hand panel before clicking on USART3. I made sure the mode was set to asynchronous and in the “NVIC settings” I enabled the USART3 global interrupt. I also verified the following parameters in the “Parameter settings”, the Baud rate being 115200 and the word length being 8 bits, making sure parity was set to none and that stop bits were set to 1 (see figure 5.9). I hit the gear icon, and it was set up.

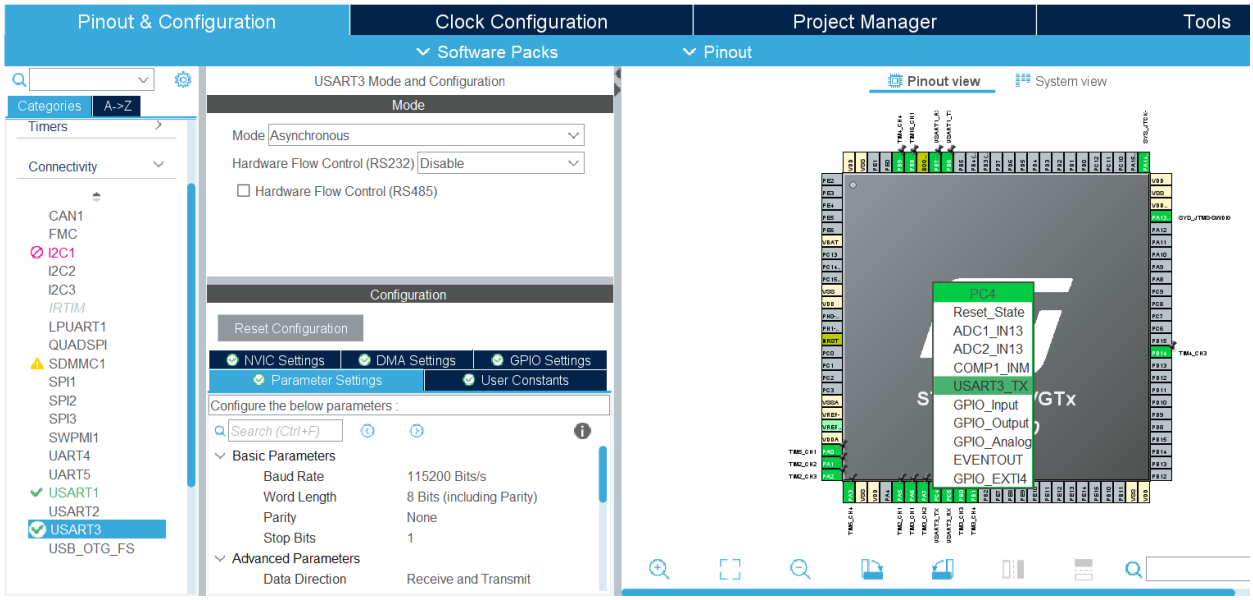


Figure 5.9 UART configuration panel (left) & pin setting (Right)

My frame for UART consisted of 8 bits with a single start and stop bit and 6 data bits.

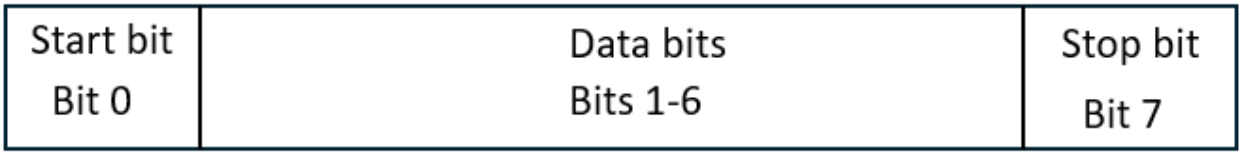


Figure 5.10 UART frame

The code implementation has huart3 as an external type definition and uses two main functions. One was to transmit data, the other to receive it. My code worked so that it had a while(1) loop that would wait for a command from the RPI and based upon the command it would act. Messages are defined above the while(1) to be sent to the RPI (see figure 5.11). Above the messages the receive buffer is declared alongside a message variable. The buffer is a width of 1 as it only receives a number.

```
char rx_buffer[1];
uint8_t msg = 0;

//Messages for the RPI
char tx_buffer[] = "Command executed\r\n";
char tx_err_buffer[] = "Invalid number sent\r\n";
char tx_start_buffer[] = "Starting the RPI\r\n";
```

Figure 5.11 Messages for the RPI

There is a case statement that switches on the command received and if it is a one for example, then it would go forward by a step. A two would be to go back etc. The HAL_UART_Receive() function is used to receive data from the RPI and takes multiple arguments [8]. It takes the UART handle (huart3 in this case), a pointer to the data buffer, the size or number of elements and the timeout [8]. The timeout waits infinitely until a message is received. I wanted the function to be blocking as my code to control servo motors should not run unless a message has been received. The msg variable is used to store the message using the atoi function. Atoi stands for ascii to integer and is used to convert the string that the message is in to an integer for the switch case logic (see figure 5.12).

```
HAL_UART_Receive(&huart3, (uint8_t*)rx_buffer, sizeof(rx_buffer), HAL_MAX_DELAY);
printf("Received: %s\r\n", rx_buffer);
//Read code from atoi
msg = atoi(rx_buffer);
//Reset before looping again
memset(rx_buffer, 0, sizeof(rx_buffer));
//Case statement to determine movement
switch (msg) {
//Forward - prerequiste vals and writes
case 1:
    printf("1 received, going forward\r\n");
```

Figure 5.12 Switch case logic upon receiving a message

HAL_UART_Transmit() takes the same arguments as the other UART function [8]. I chose to implement a version of this at the end of each case statement to notify the RPI that a command had been executed (see figure 5.13).

```
HAL_UART_Transmit(&huart3, (uint8_t*)tx_buffer, strlen(tx_buffer), HAL_MAX_DELAY);
```

Figure 5.13 Transmitting a command complete message to the RPI

On the RPI side, the interface had to be enabled for communication to occur. I had to enter the command “sudo raspi-config” in the terminal I was using and go to “interface options” followed by “serial” and finally “enable serial port” [9]. I had to restart the system with “sudo reboot” and serial communication was enabled.

Two libraries make the UART interface operate on the RPI. This was done referencing an article showing how to communicate between an RPI and STM32 microcontroller [9]. I did not have to edit the code much to get it working, but I did research it to understand what was happening. The file imports time and uses time.sleep() to delay the number of seconds entered within the parentheses [10]. PySerial is used to create a serial object that sets the port, baud rate, parity, stop bit, byte size (data length) and a timeout flag [11]. The frame from the RPI to the STM32 matches the UART frame it sends (see figure 5.14). The code works so that a serial object is created and used to write to and read from the serial port as required. A keyboard interrupt can close the port as ser.close() is called and the file stops [11]. This file was mainly used for testing purposes, such as trying to figure out the movement of the legs. This script was later made into a serial communication function in the final Python code.

```
# Configure serial port
ser = serial.Serial(
    port='/dev/serial0',  # Default UART port on GPIO 14/15
    baudrate=115200,
    parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE,
    bytesize=serial.EIGHTBITS,
    timeout=1
)

try:
    while True:
        # Send message
        ser.write(b"1")
        print("Sent: 1")
```

Figure 5.14 Serial communication from the RPI

6 Part II: The Raspberry Pi 4 Model B

6.1 What is the Raspberry Pi?

The RPI is the brains of the SaRQ. It communicates with the STM32, ordering the way in which the SaRQ should move, while making decisions based upon the readings it receives from other sources. The RPI is not a microcontroller similar to the STM32IOT1A, but a full computer, allowing you to power it over 5V via USB-C connector, attach a monitor via one of two micro-HDMI ports and plug in a mouse over one of the two USB 3.0 ports or the two USB 2.0 ports [12]. The RPI has many more hardware features, but most importantly of all it contains a system on chip called the BCM2711 [12][13]. The BCM2711 has a Quad-core Cortex-A72 as a central processing unit and up to up to 8 GB LPDDR4-2400 SDRAM. There are a few peripherals that the BCM2711 contains such as timers, GPIOs, I2C masters, SPI masters, PWM, UARTs etc [14]. The RPI is connected to the Pi cam and runs YOLO for human detection. Each image the Pi cam reads in has the YOLO model used on it before it is sent to a browser to be seen live over the MJPEG server the RPI hosts. This server is an option for SARP if they want to see the SaRQ's

live view. The RPI code is written in Python and in the case a person is detected for ten frames, an SMTP server is started, and ten images are taken and attached to an email that SARP receive. This email has never in testing had blurry attachments, but in the case it does, the MJPEG server can be used as a backup for SARP to see if a human was detected. Testing was used to see the confidence rating the model should run to balance detecting a person in debris or in a complex scene, while also being accurate. An IK file was run on the RPI as a tool to model the motion of the SaRQ in a set co-ordinate system while constructing the walking animation of the project.

6.2 Inverse Kinematics

IK is the use of equations to determine the motion a robot must undergo to reach a point in space [16]. Breaking that definition down, you can understand that a robot uses IK to go from where it currently is to a new point. In the case of the SaRQ, this requires mathematical formulae to calculate the angles that each servo must undergo to move to a desired point. I wanted to implement a Python file that would calculate the angles required to reach a point in space. This way, I could map the motion of the SaRQ in a co-ordinate system and more accurately predict the motion. The idea was to feed the animation for the motion of the SaRQ into this co-ordinate system to then find out the movement to a high accuracy and use the file as a tool for creating a movement animation. Let me also preface by saying the terms hip, shoulder and elbow will be used throughout the report and are defined as follows: the hip servo rotates the leg from left to right i.e. the X axis when looking from above, when viewing from the side, the shoulder servo rotates the leg up and down (along the Z axis) and the elbow servo brings the leg in and out (along the Y axis).

6.2.1 First Attempts & Rabbit Holes

While researching for the leg design of the SaRQ, I found various ways that IK was implemented. There were different methods, but overall, the idea was the same. Firstly, you must create triangles and use trigonometry to calculate the angles. The code would receive the desired X and Y and Z co-ordinates and after running calculations, the answer for the three angles would be returned. When designing the legs (see Designing the SaRQ) I researched videos on different designs and implementations. Given that the first video did not exhibit a

stellar implementation (i.e. the turtle motion, dragging across the floor), I focused on the second video which explained the concepts in depth.

The video explained the mathematics quite well but did not clarify everything in my opinion. To create equations to map the angles, you need to create two triangles, the first connects the tip of the leg or co-ordinate you want to move to with the elbow servo in the middle of the leg, and the shoulder (S2 and S1 respectively) [15]. The second is a right-angled triangle connecting the co-ordinate at the tip of the leg, to the line going through the shoulder (see figure 6.1) [15].

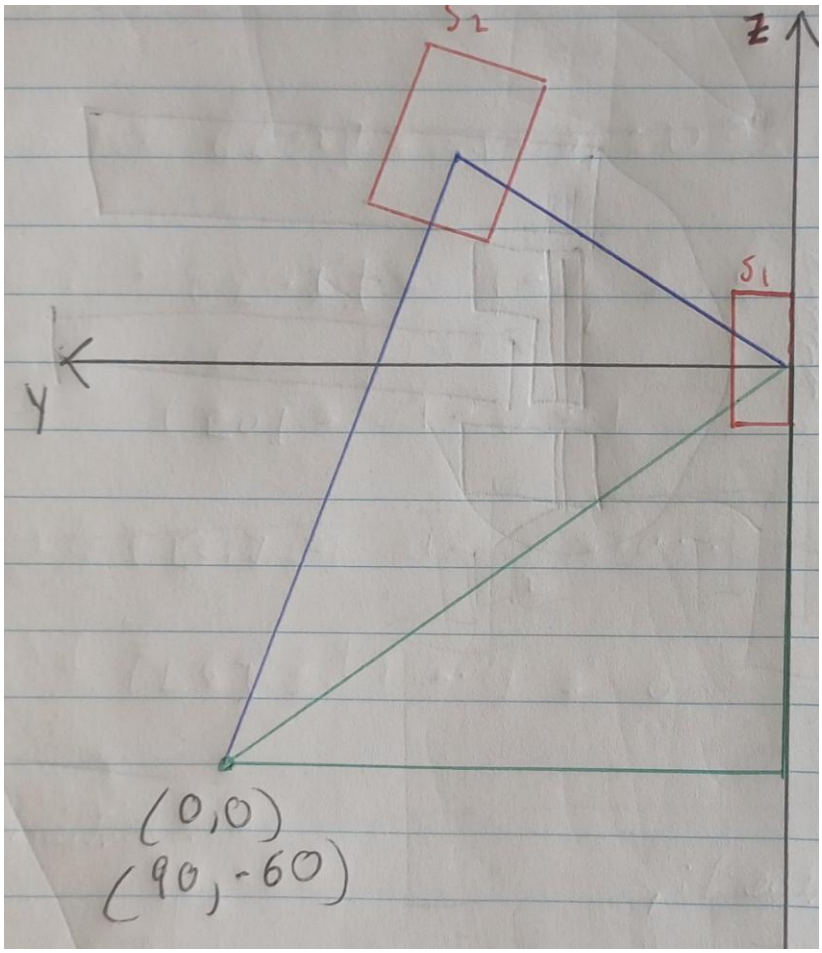


Figure 6.1 The triangles for the IK mathematics

From these triangles, we can calculate the angles needed for each servo motor to move in the Z and Y direction [15]. The first triangle or T1 has two known sides since they are the length of the upper and lower leg segments [15]. The only unknown is the other side's length which is the

hypotenuse of the right-angled triangle or T2 [15]. The hypotenuse is found using the Pythagorean theorem, angles are found using SOHCAHTOA for right angled triangles and the cosine rule used for non-right-angled triangles respectively [15]. SOH CAH TOA is a means of calculating an angle based on where it is in the triangle. For example, SOH stands for sin(A) = opposite/hypotenuse, CAH stands for cos(A) = adjacent/hypotenuse and TOA stands for tan(A) = opposite/adjacent. The sides are shown in figure 6.2.

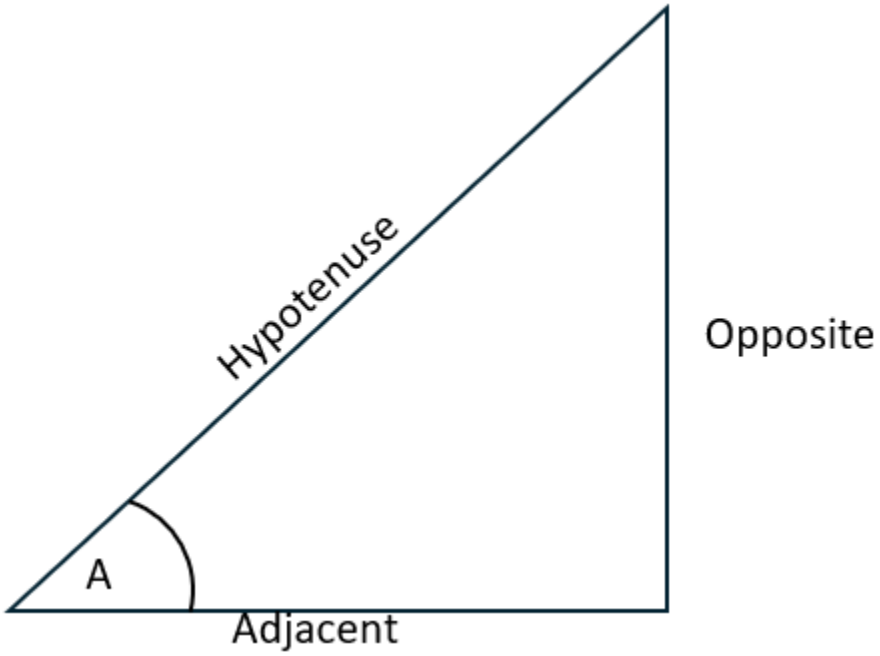


Figure 6.2 Sides of a right-angled triangle

The cosine rule is $a^2 = b^2 + c^2 - 2bcCos(A)$. Given three sides of a non-right-angled triangle, you can calculate an unknown angle. The lower-case letters are the sides and angles are represented via capital letters.

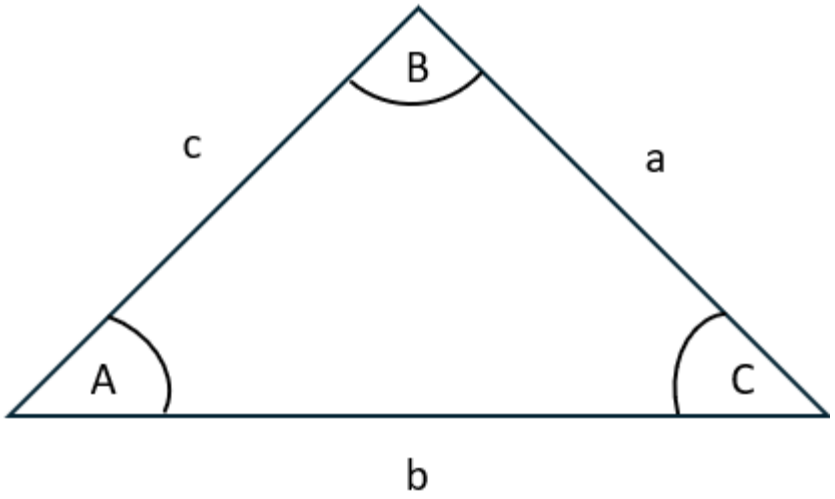


Figure 6.3 The cosine rule

For simplicity, I will focus on another video when showing the mathematics in action as there were issues with this video. Firstly, everything was relative to offsets which were incorporated into the formulae given but could be fixed by mounting the servo motor differently instead [15]. Servo motors have directions of movement and may be attached at a specific angle to enable the range of motion desired (see figure 6.4).

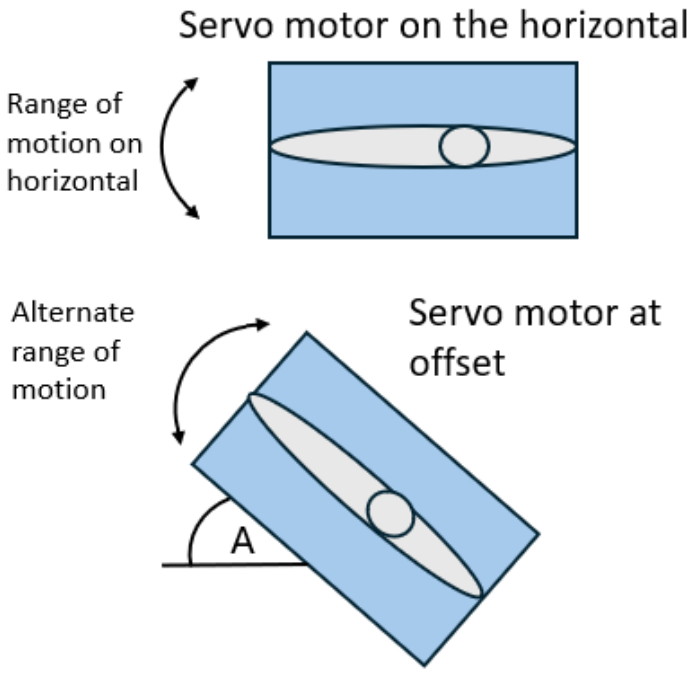


Figure 6.4 A servo motor at an angle versus the horizontal

For example, the servo motor shown above is not attached relative to the horizontal but at an angle which must be known if you are to follow the guide correctly. In the guide, rather than attach the servo at the offset, the offset was inserted into the code which complicated it further [15]. Imagine finding the angle A but remembering that 90 must be removed by A at the end to compensate for this offset. It is more complicated to think about in your head and could be easily rectified in the hardware rather than complicating the code. This is not a major issue by any extreme, but it was relevant to note as paired with the co-ordinate offset the code and hence mathematics becomes more strenuous for no reason.

The co-ordinate system is relative to the shoulder servo motor, which doesn't makes sense logically, presenting an issue. Rather than beginning from the centre of the shoulder servo (S1 on figure 6.1), it starts at an offset on the ground [15]. In the diagram, (0,0) is (90,-60) and means that any co-ordinate must have the offset added which is just overtly complicated in my opinion [15]. The offset for the co-ordinate was included as the leg tip was the origin and the tip would begin at (90, -60) when the robot stood. If you are moving the tip of a leg 100 millimetres (mm) in front from the tip's horizontal, that is not (100, 0) but (190, -60) note -60 is whatever the distance from the origin that the leg is down. Having the leg's horizontal be negative means that the angle A calculated above was initially in the video A = A -90 but due to this negative it is A = A –(-90) which is A = A + 90 [15]. I was confused at this point in my research, and worse was the fact that my code was giving me mathematical errors. This was due to another unforeseen issue which will be addressed later, but thanks to these offsets and double negatives, I wasn't sure whether the issue was my fundamental understanding of the mathematics or not. I was confident in my mathematical background before this, having studied Physics and Applied Mathematics. The basic formulae involved in this were trivial compared to this, but the logic was the tricky part.

6.2.2 The Mathematics

Before I moved onto another source for creating my model, it should be noted that I did find the repository that the first video linked, to try and trace the issue I was having. There were several issues with trying to pinpoint where I was going wrong based off this. Firstly, the code was written for an Arduino which made it harder to interpret compared to in built Python

functions for mathematical operations [17]. I have programmed in Arduino and can read it, but the code on the repository had been edited since that video was released and did not look familiar [17]. There were segments with the same formulae that I had that matched perfectly, and the additional code seemed to be an increased scope due to another video the channel made [17]. I decided to follow another tutorial on creating IK but this time I was keenly interested in comparing the formulae with what I currently had and hoped to find a guide without offsets built into it.

The tutorial I found had the same base fundamentals as the other one, there were two triangles, and the same formulae (Cosine rule, SOHCAHTOA and Pythagorean theorem) were used [18] (see figure 6.5).

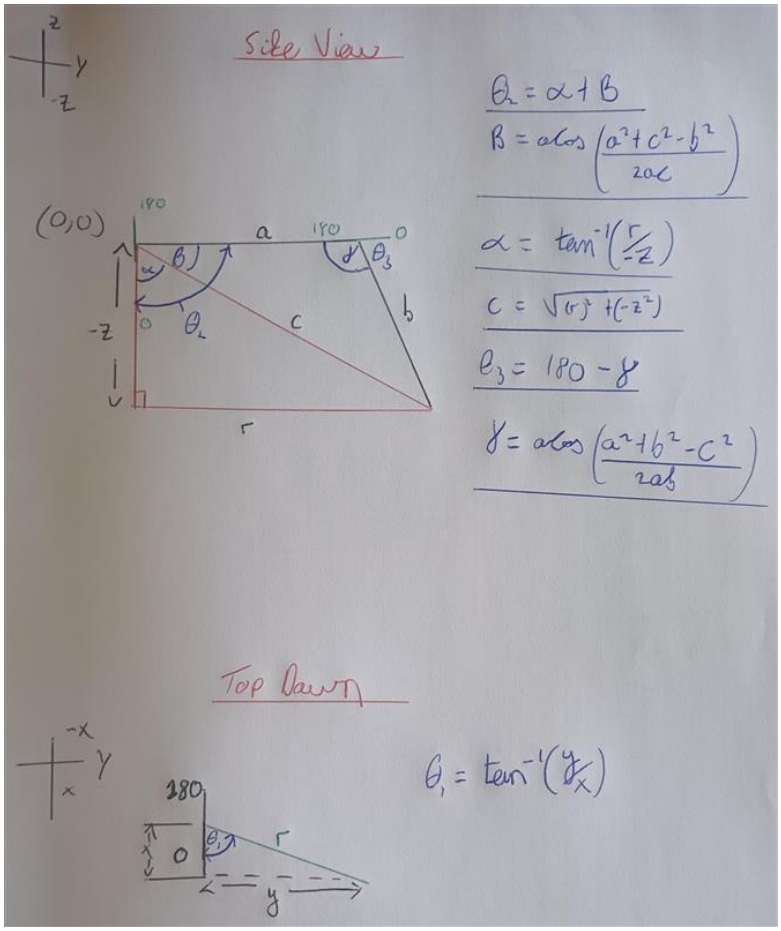


Figure 6.5 Triangles and equations associated with the second video

Now that we are focusing on the video that I based my own code on, I can go further into depth on the way each angle is found. The video begins using labelling for angles or the servo rotation which avoided the error of the last video regarding clarity [17]. In the diagram above, the angles 0 and 180 are shown to indicate the range of motion for all three servo motos [18].

From the top-down view, the servo motor has 180 on the left and 0 on the right (if you are standing behind the servo motor) [18]. The side view has the shoulder servo with 180 on top and 0 on the bottom and the elbow has 0 being directly opposite the origin while 180 would bring the servo to overlap it [18].

This is where the organisation is important as the labels for the X,Y and Z axes as well as the sides of the triangles is the next step (see top left of image above). Beginning with the top-down view, the X axis is the vertical and Y the horizontal [18]. The Y axis is the horizontal in the side view and Z the vertical [18]. $Theta_1$ is the angle for the hip, $theta_2$ the shoulder angle and $theta_3$ the elbow angle [18]. Side A and B are known, being the upper leg length and lower leg length respectively [18]. It should be noted that in the video there was an offset between the origin and the triangles due to the nature of the design which mine does not require [18]. The origin also starts at the shoulder servo which means the co-ordinate offsets were not required for this code. $Theta_1$ is always a part of a right-angled triangle and is equal to the opposite side over the adjacent or using SOHCAHTOA we can say $Tan(theta_1) = O/A$ or Y/X. To solve for $theta_1$ it becomes $theta_1 = Tan^{-1}(Y/X)$ [18]. The video then introduced atan2() [18]. This is where a fundamental issue with the code was solved for me. This function knows what quadrant the point is in and calculates the offset to match it while also avoiding divide by 0 errors [18]. I did not know it at the time, but this is where the IK was going awry for me.

$Theta_2$ is found to be alpha + beta which are two variables that need to be calculated themselves before $theta_2$ can be found [18]. Using the same formula for $theta_1$ (SOHCAHTOA) we can get $alpha = tan^{-1}(d/-Z)$ [18]. D is the distance out from the origin and -Z is the down from the origin which are both known [18]. Z is negative since the origin is (0,0) and travelling down is negative in this case [18]. The cosine rule is employed to find beta [18]. The cosine rule is $b^2 = c^2 + a^2 – 2acCos(beta)$ given that we are trying to find beta [18]. Making beta the subject of the

formula, we get beta = aCos((a^2 + c^2 - b^2)/ 2ac) [18]. We know almost all the variables here since the side a is the upper leg, b is the lower leg but we don't know side c. For that we use the Pythagorean theorem or A^2 = B^2 + C^2 [18]. A in this case is side c while -Z and r are B and C respectively [18]. R is the value of the square root of X^2 and Y^2 as the top-down view shows that r is the hypotenuse of a right-angled triangle [18]. X and Y are known and hence beta can be found leading to theta$_2$ (see figure 6.5)[18].

Theta$_3$ combined with gamma is 180 degrees [18]. We can use the cosine rule once more to calculate gamma and hence theta$_3$ [18]. c^2 = a^2 + b^2 – 2abCos(gamma) [18]. Making gamma the subject of the formula, we get gamma = aCos((a^2 + b^2 - c^2)/ 2ab) [18].

6.2.3 The Code

So, how does this look in code? It is eerily similar to the mathematical formulae, and the code can be viewed below.

```
"""Module example for only inverse kinematics"""
from math import acos, sqrt, pi, atan2

#Length upper and lower globals
LEN_U_L = 60
LEN_L_L = 112

#Inverse kinematics calculations
def servo_calc(x_co_ord, y_co_ord, z_co_ord):
    #Distance to tip
    dist_d = sqrt(x_co_ord**2 + y_co_ord**2)

    #Hypotenuse distance
    hypotenuse = sqrt(dist_d**2 + z_co_ord**2)

    #Angle calculations
    hip_a = atan2(y_co_ord, x_co_ord) * 180/pi
    alpha = atan2(dist_d, -z_co_ord) * 180/pi
    beta = acos((LEN_U_L**2 + hypotenuse**2 - LEN_L_L**2)/(2*LEN_U_L*hypotenuse)) * 180/pi
    shoulder_a = alpha + beta
    elbow_a = 180 - acos((LEN_U_L**2 + LEN_L_L**2 - hypotenuse**2)/(2*LEN_U_L*LEN_L_L)) * 180/pi

    print(f"Theta 1 (hip) is: {hip_a} Theta 2 (shoulder) is: {shoulder_a} Theta 3 (elbow) is: {elbow_a} ")


#X, Y Z is passed or hip, horizontal and vertical
servo_calc(0,89, -34)
```

Figure 6.6 IK angle generation code

The formulae were able to be written as is and a function was created called servo_calc to pass the desired co-ordinates into. This was done so that when I used the Desmos tool, it was easier to input the co-ordinates. The code calculates the servo angles before printing them to the console. The upper and lower leg lengths are input as global variables and functions such as $\cos^{-1}$ (although in code it is acos) for use in the mathematical calculations.

6.2.4 Testing the IK: verification & Servo Motor Calibration

After creating the Python file and receiving no errors on output, the next stage was to make sure that it was giving valid outputs for the inputs given. The guide urged me to use the tool Desmos inverse kinematics calculator 2 to verify the validity of the outputs (see figure 6.7)[19][18].

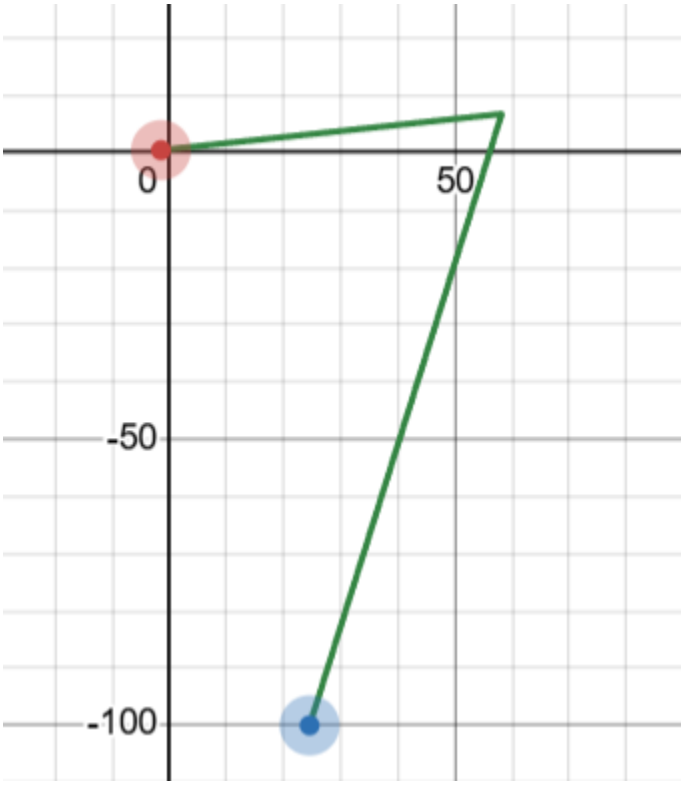


Figure 6.7 Desmos IK calculator with my upper and lower leg lengths input [19]

The means of verifying the outputs (before using hardware of course) was to predict the angles that the leg would be in given the Desmos tool [18]. You can see in the image that the angle for the shoulder should be around 100 degrees and for the elbow it may be 40. If you insert the co-

ordinates of the tip from the Desmos tool into the file and get the outputs of 100 degrees and 40 degrees respectively, you can assume your model is relatively accurate [18]. I tested the accuracy by moving the angle of each leg and seeing if the output would match what I expected. I made sure that my file followed closely to what was expected, setting the legs to various positions before inputting the co-ordinates into my Python file.

The next stage of development focused on testing the servo motors, making sure they rotated in the correct direction and that the angles were as they were supposed to be (i.e. 180 degrees on top instead of 0 degrees). Once the servo motors were calibrated, I screwed the servo horns in to secure them and began to build the legs (see the section Building the SaRQ for more detail). I tested the IK file with my new legs to verify what I had seen with the Desmos tool was still true for the hardware in front of me and found that for the direction of motion of one leg I had set the angle of rotation incorrectly. The elbow should have 0 degrees as directly perpendicular to the shoulder servo motor. Instead, I found that I had attached the servo motor incorrectly at first (see figure 6.8).

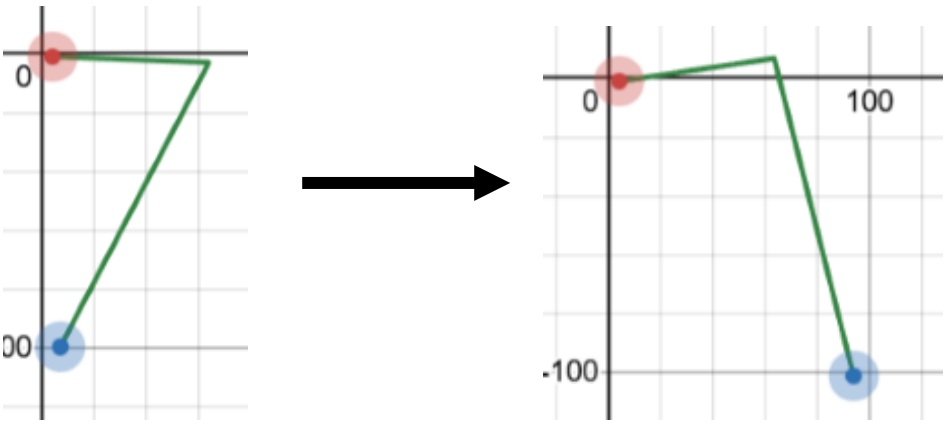


Figure 6.8 The inputs for testing the hardware followed the IK model

Figure 6.9 The hardware output not matching the model

I rectified the mistake by attaching the servo motors correctly. It is apparent from figure 6.10 that the leg starts in a closer position and finished closer to the model. This is where the reliability of the servo motors became an issue as I believe my model is accurate but due to the cheap servo motors, they are not as accurate given the inputs they receive.

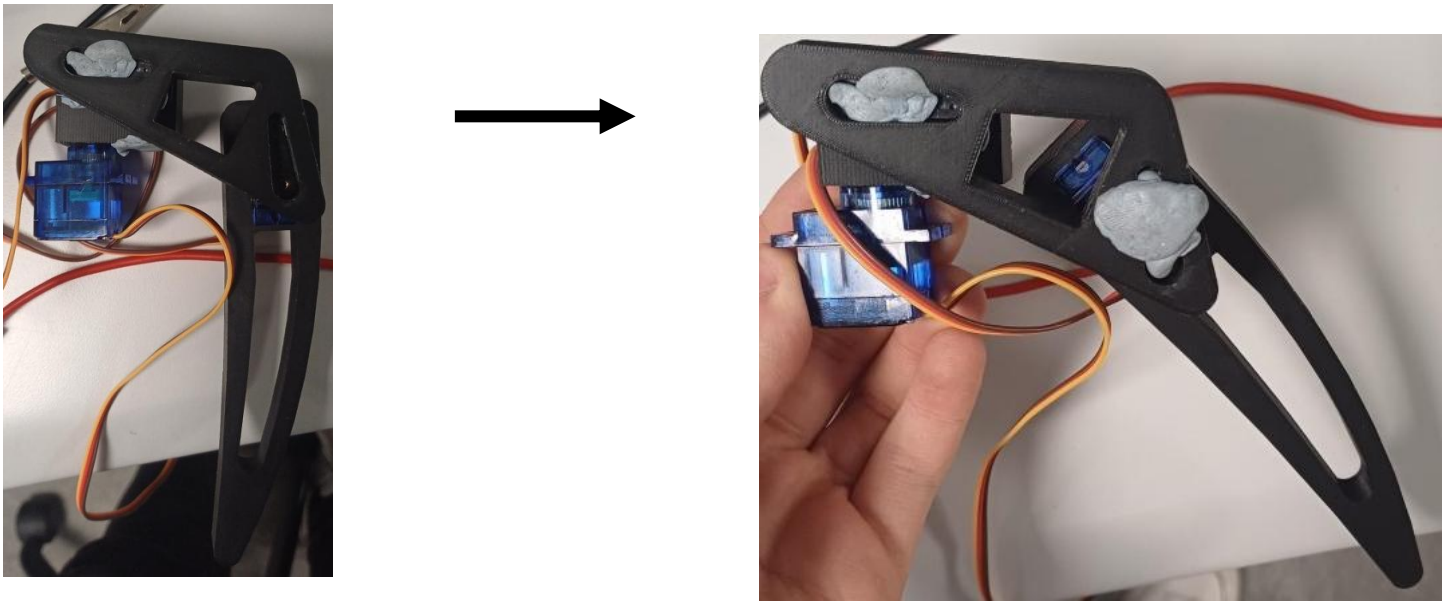


Figure 6.10 The improved hardware output

I tested the accuracy of the IK model to a higher standard by setting the Desmos tool to move with the leg in a set starting position and then a set distance apart. In the example below the initial position and second differ by 10mm in the Y axis (Y in the IK model which would be the X axis in a generic mathematical one).

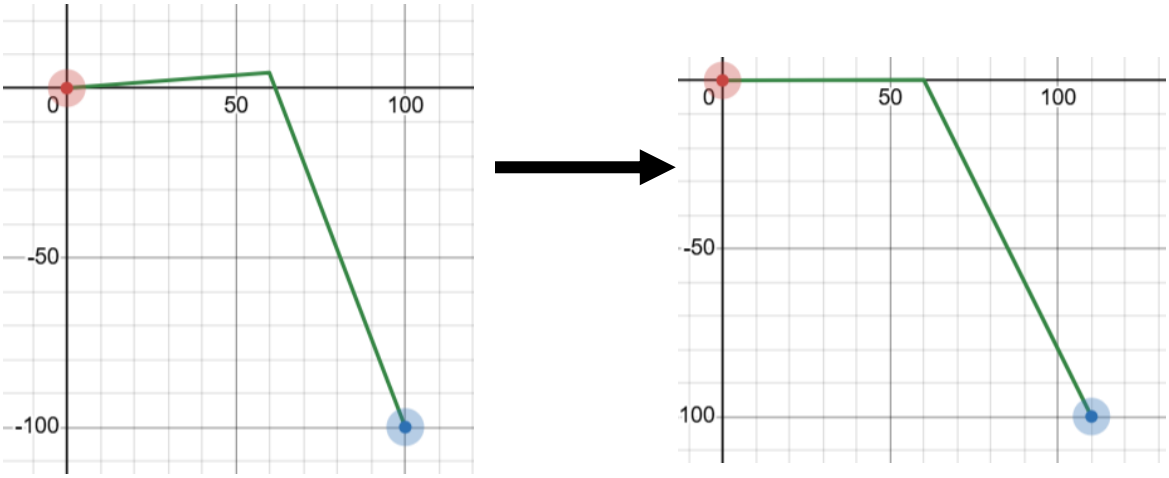


Figure 6.11 The model test in the Y axis

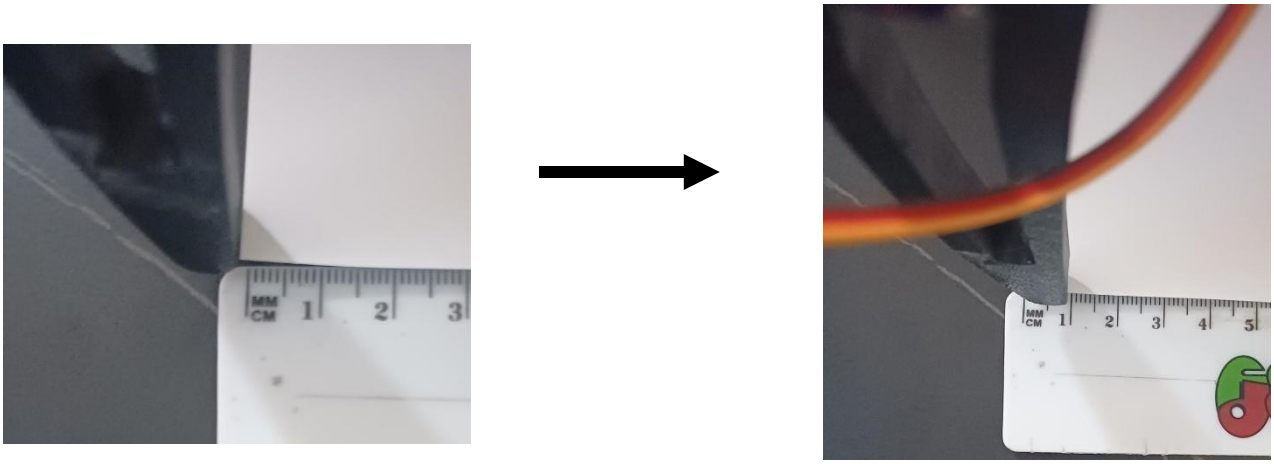


Figure 6.12 The hardware output for a 10mm movement

The preliminary results boded well, with an accurate movement as desired. This was not the case when the leg was set to move 39mm (the max the model allowed) in the Y axis, leading to an accurate reading of 39mm moved, but the leg rose 10mm in the Z axis as well which it should not have. This accounts for a reason why the SaRQ could not take large steps as these inaccuracies would be perpetuated the larger the steps taken were. Note that the same test was performed in the Z axis the front left leg was raised 10mm high which was accurate and moved in a singular axis but when moved 39mm upwards, the leg moved out in the Y axis by roughly 5mm. While I attempted to get the servo motors to have the smallest resolution possible, the attachment on the gears for the servo horn is not a small resolution. This means that the servo motors can move to where I set, but not from exactly where I want them to start. This could be a culprit for errors. The servo motors are cheap, and I believe that no matter how precise the resolution is in code, in an application such as this, the error is more pronounced when tested. I believe my model is accurate and that the servo motors themselves are likely the issue along with the mounting of the servo motors on the SaRQ which are rigid but can be wiggled slightly in their clamps.

6.3 Detecting Humans

The SaRQ cannot be used in search and rescue scenarios without the ability to detect humans. In the case of this project, a Pi cam was used as a means of seeing people in the environment the SaRQ was deployed in. YOLOv11 was implemented to detect humans, while an MJPEG

server was used to stream a live feed of the camera to the user. If a human is detected, the user is alerted via email with the first 10 images of a human attached as a means of combatting blurry images.

6.3.1 What is YOLO?

What is YOLO? You only look once or YOLO is an object detection model designed for real-time detection [20]. It is faster and more accurate than other models, predicting bounding boxes and class probabilities simultaneously from the entire image in one go [20]. YOLO is a convolutional neural network or CNN, which is a deep learning model designed to process data such as images [21]. The architecture of YOLO has 24 convolutional layers, four max pooling layers and two fully connected layers [22].

What do these terms mean though? Starting with convolutional layers, they apply operations to input images using filters or kernels to detect features [21]. These features can be a range of things such as edges, textures, or something more complex [23]. Convolutional layers are made up of four key components. These include filters, stride, padding and an activation function [24]. The stride is the step size the filters move across the input data [24]. Larger strides result in smaller output feature maps and hence it is quicker to compute [24]. Padding is when zeros or other values are added to the output to control the width and height of the output [24]. A non-linear activation function is applied to allow the network to learn relationships between the data [24]. An example of an input image of 32x32x3 with a convolution layer with ten 5x5 filters, a stride of 1 and 'same' padding will produce an output feature map of size 32x32x10 [24]. Each of the 10 filters detects different features in the input image [24].

A max pooling layer selects the maximum value to reduce the size of feature maps while maintaining the most important information [23]. There are other types of pooling, but max pooling keeps the pixels with the highest activation [23]. For example, imagine a matrix of pixels, which you split up into 2x2 matrices i.e. 2x2 max pooling with a stride of 2 (see figures 6.13 to 6.15) [23].

8	1	3	9
7	2	1	1
3	5	8	4
5	1	6	1

Figure 6.13 The starting image

8	1
7	2

3	9
1	1

3	5
5	1

8	4
6	1

Figure 6.14 The image split-up into 2*2 matrices

8	9
5	8

Figure 6.15 The reduced matrix

Max pooling reduces computation with smaller feature maps, leading to fewer parameters and faster training. It improves robustness as if the object present in the image is moved slightly, the model still detects it with max pooling and noise isn't memorized i.e. overfitting is avoided thanks to the reduced size of feature maps [23].

Convolutional layers take the image and make feature maps, max pool layers reduce the feature maps, but where do fully connected layers fit in? Well, this layer has every neuron connected to every neuron in the previous and subsequent layers [22]. The fully connected layers are usually the last layers after convolutional and max pooling, taking the feature maps and making final outputs or predictions (see figure 6.16) [22].

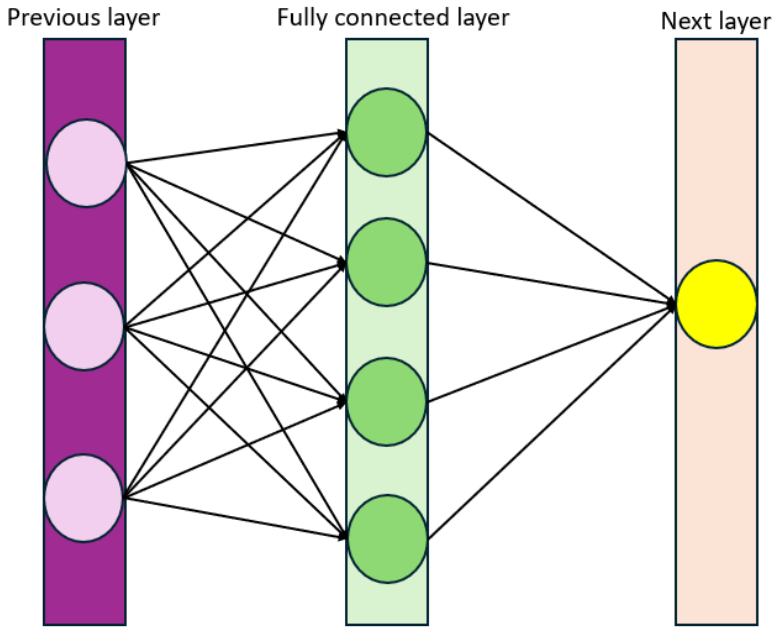


Figure 6.16 A network containing a fully connected layer

A fully connected layer consists of neurons which connect to neurons of the previous layer and take inputs before sending outputs to the next layer [25]. Each connection between the neurons has a weight associated with it similar to the influence of one neuron on the other [25]. There is also a bias which adjusts the output with the weighted sum of inputs [25]. Finally, there is an activation function to introduce non-linearity to the model to learn complex patterns [25].

6.3.2 How Does YOLO Detect a Person?

YOLO detects a person (or any object) via a four-step process. The image is split into a grid of cells, equal in shape (see figure 6.17) [22]. Each cell in the grid must predict the class object and includes a confidence value or the chances that the model thinks that its prediction is correct [22].

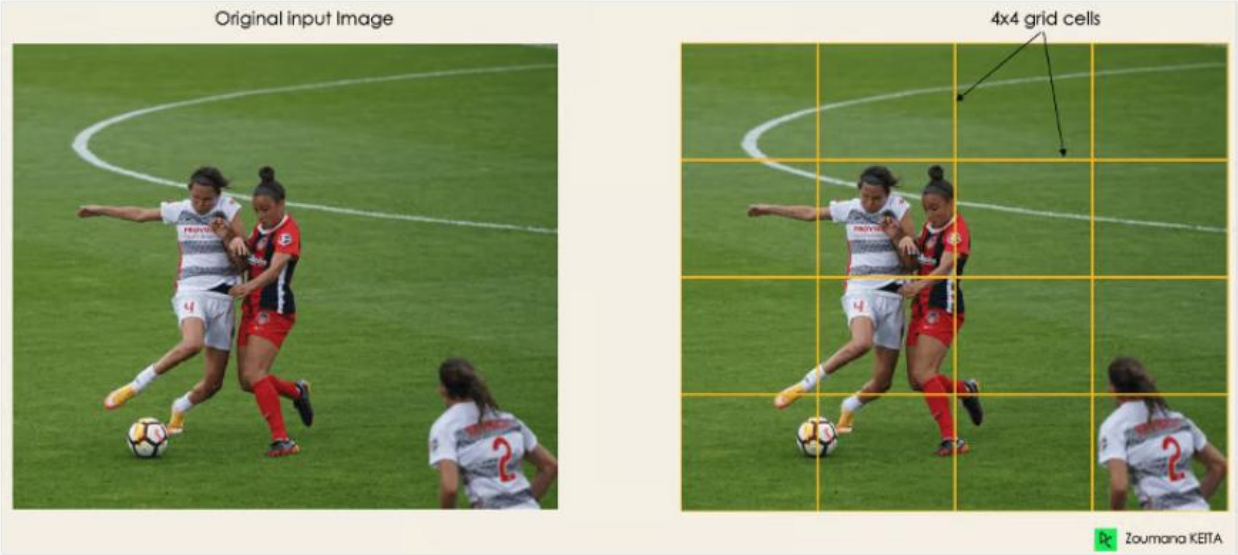


Figure 6.17 An example grid with people [22]

All the objects are highlighted within the image, creating boundary boxes based upon the grid cells [22]. The attributes of these bounding boxes are found with a single regression module, where Y is the final vector representation for each bounding box. Pc is the probability score of the grid containing an object i.e. all the highlighted grids have a pc>0 (see figure 6.18) [22].

Y = [pc, bx, by, bh, bw, c1, c2]

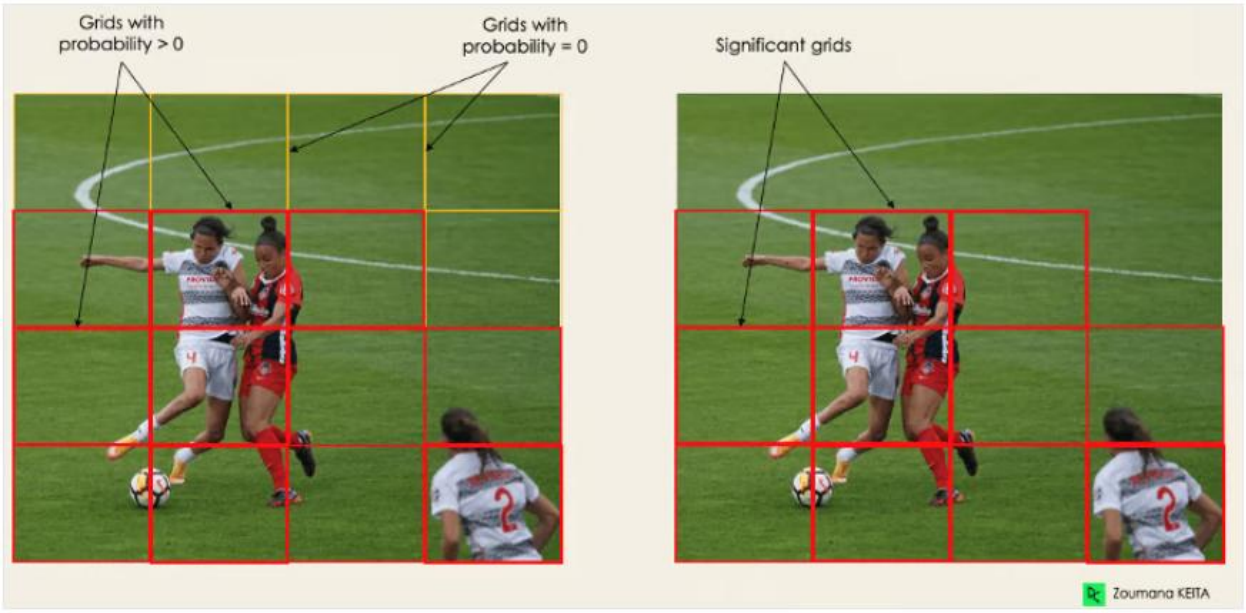


Figure 6.18 Images with cells containing objects are highlighted [22]

bx, by are the x and y coordinates of the centre of the bounding box and bh, bw are the height and width of the bounding box with respect to the enveloping grid cell [22].

C1 and c2 correspond to the two classes, player and ball in this example [22]. The number of classes is not fixed and can extend based on your requirements [22].

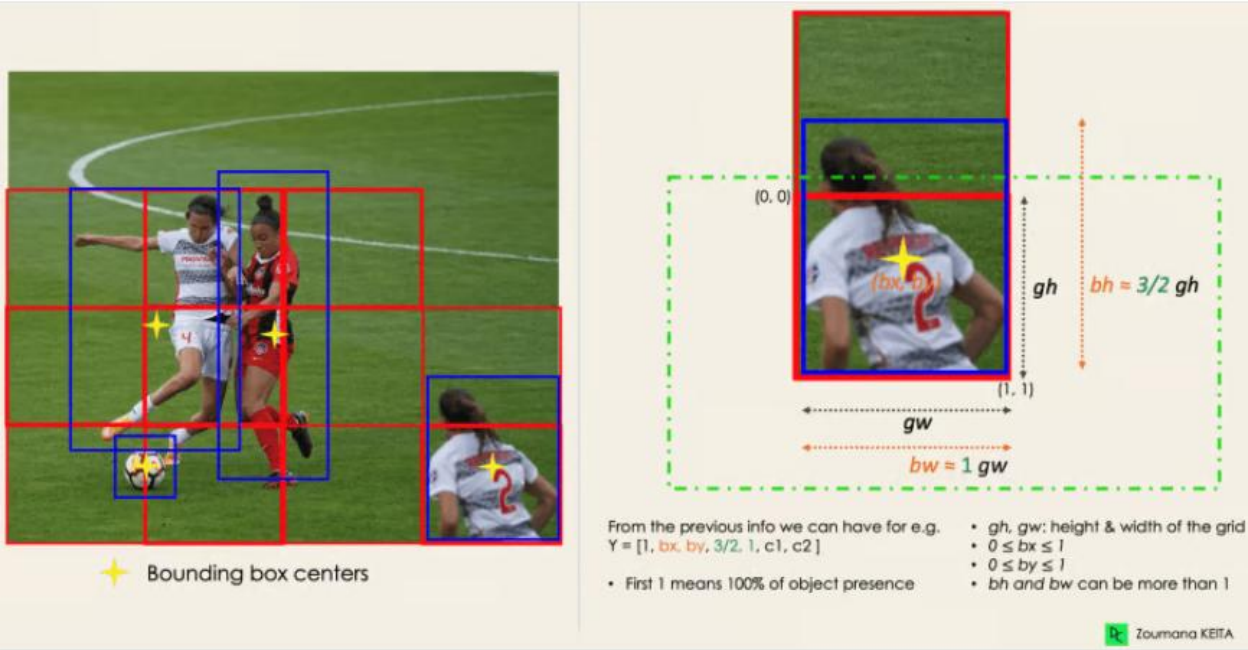


Figure 6.19 Example images with bounding boxes [22]

Intersection over unions or IOU has the goal of discarding boxes that are not relevant [22]. What I mean by this is that an object may have multiple grid box candidates and would eliminate one if needed [22]. IOU is a value between 0 and 1 and is user defined [22]. YOLO computes the IOU of each cell i.e. (the intersection area) / Union area and it ignores the prediction grid cells with an IOU less than the threshold (see figure 6.20) [22].

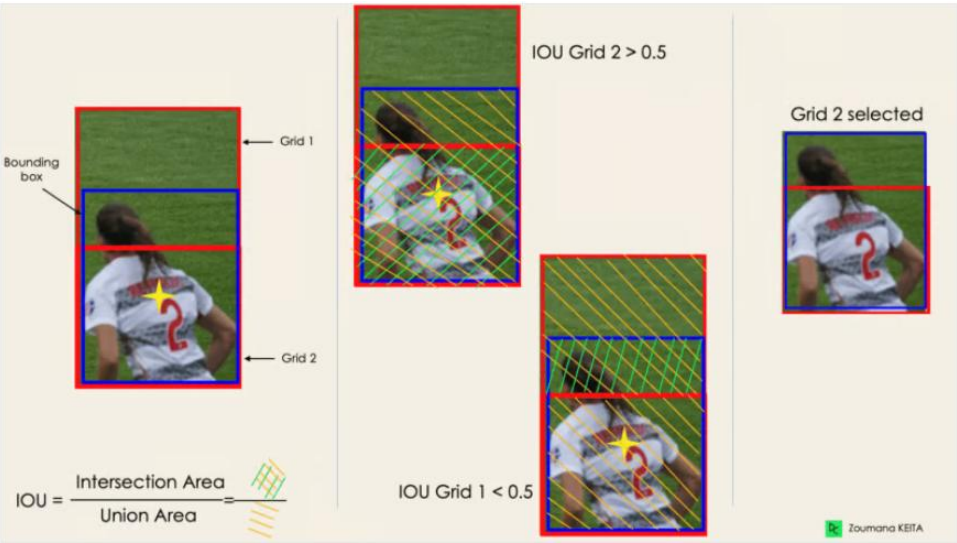


Figure 6.20 Intersection over union is implemented [22]

The final step is non-max suppression or NMS [22]. NMS is used to reduce noise as if the IOU threshold leaves multiple boxes included it needs to be fixed [22]. NMS leaves only the box with the highest probability detection score [22].

6.3.3 Why YOLO?

I've covered the background of YOLO in detail, including how a CNN works, the architecture of YOLO specifically and how the algorithm detects a person or object. The question remains, why did I choose YOLO over other algorithms? I performed research into other models when selecting a means of detecting people. I knew I needed something already capable of detecting objects in busy environments such as those in disaster zones. I knew my hardware was a restriction, but I wanted something that would be fast and yet not computationally too expensive. In the end I compared YOLO, single shot multibox detector or SSD and the region-based convolutional neural network (R-CNN) family.

R-CNN uses a two-stage pipeline in which region proposal is used via selective search before feature extraction and classification with boundary box regression is used [20]. In short, each region is analysed which gives accurate results but takes a lot of time and computational power [20]. This algorithm later evolved into fast R-CNN where a single forward pass occurred across the image among other improvements leading to a faster model [20]. This was improved even further into faster R-CNN which is one of the most accurate two stage detectors [20]. Any high precision applications are use cases for the R-CNN family such as industrial quality inspection and medical image diagnostics [20]. The downside of course, is the fact that due to slow inference (time to analyse data and make predictions) and feature extraction is computationally expensive [26][20].

SSD works in a similar way to YOLO, performing a single forward pass to create bounding boxes and performing classification [20]. This makes it faster than the R-CNNs that require two stages [20]. It is also great at balancing accuracy and speed [20]. There are disadvantages to note, such as weakness detecting small objects in complex scenes and scaling to complex scenes [20]. It also has less accuracy than some YOLO models [20]. Those disadvantages are good to keep in mind given that the task of the SaRQ is to navigate in complex scenes, detecting people potentially on the ground or amid debris. SSD would usually be used in real-time vision for robotics, edge artificial intelligence projects and open-source academic research [20]. Note that robotic vision is something that I was looking for when researching this.

You have heard the case for YOLO at this point, but the main strengths were the fact that it is fast, accurate and versatile [20]. It is fast due to predicting bounding boxes and class probabilities in one go, where other models would use two stages [20]. It has several applications, including object recognition and path planning in dynamic environments for robotics, edge computing (allowing smaller versions of the model to run on embedded devices) and drone-based surveillance to name a few [20]. It is apparent that the first reason suits my project quite well and it boded well that embedded devices were mentioned as well. You will see later, using tutorials I was able to optimise the model for the RPI.

The reason I discounted the R-CNN family was due to the speed and computation required. I needed my project to be close to real-time and the RPI was not designed for heavy computation. SDD was close to being my choice, but the fact that it does not like complex environments meant that I couldn't risk using it given the scope of my project. This left YOLO as the clear winner and needed to be incorporated into my project.

6.3.4 Adding YOLO to my project

Most of the code used for the purpose of detecting humans was taken from a document describing setting up the hardware, libraries and gave sample code for object detection [27]. The operating system (os) that was installed on the RPI for the tutorial (Bookworm) was older than the current latest (Trixie) and since I used the latest os, issues began to crop up [27]. I had to create a virtual environment (venv) which keeps libraries that you install, separate from the system libraries to avoid conflicts [27]. The issue with installing Trixie as the os was that the main library you would install (called Ultralytics) was not suitable for a Python later than 3.12 [28]. The system Python on Trixie is later than that and initially I tried installing a new system Python that was older than the current one to fix this issue but alas I still ran into errors. The main issue was that the camera was using one version of Python and the virtual environment was using another. This in turn meant that they couldn't directly communicate and I installed Bookworm to fix the issue.

The code began in a simple format, beginning with initialising the Pi camera before loading the YOLO model[27]. This is followed by a loop which takes each frame, runs the model on said frame and creates a boundary box around the object detected [27]. The next section of code is merely for measuring the inference of the model where once the frames per second (fps) is calculated it is drawn onto the frame for us to see (see figure 6.21) [27]. The first model I used was YOLOv8, the nano version specifically. The nano version is the smallest model, running faster but with the drawback of being less accurate at detecting objects [27]. The models have also improved over time, becoming more accurate with minimal downsides if any [27]. I used the latest version for detection which was YOLOv11.

```
# Initialize the camera
picam2 = Picamera2()
picam2.preview_configuration.main.size = (640, 640)
picam2.preview_configuration.main.format = "RGB888"
picam2.preview_configuration.align()
picam2.configure("preview")
picam2.start()

# Load the YOLO model
model = YOLO("yolo11n_ncnn_model")
```

Figure 6.21 Starting the camera & loading the model

After creating the minimal version, I focused on optimising performance. Initially I was using a virtual network computing (VNC) viewer to view the window for the object detection, but this was tough to run on the RPI, and I was getting very low fps (roughly 5). I later implemented an MJPEG server to reduce the load on the RPI and with the following optimisation, I managed to get a much faster feed. I added a script to convert the model to NCNN format, which is optimised for ARM CPUs like the RPI [27]. The script simply loads the model similar to the main script and exports it to the NCNN format with a single command (see figure 6.22).

```
# Export the model to NCNN format
model.export(format="ncnn", imgsz=320)  # creates 'yolox_ncnn_model'
```

Figure 6.22 Model conversion script

Changing the model can increase the fps by roughly four times [27]. There is an argument for the resolution of the model which affects the number of pixels in the image and more pixels leads to longer processing times as there is more for the model to check [27]. The default is 640 but that can detect objects at quite a long range, meaning altering this to 320 is better for performance [27]. In the results section of the code, you must add the resolution as an argument if you change it from the default [27]. I also added a confidence value of 0.8, meaning unless the model is 80% certain or above that an object is a human, it does not put a bounding box on it [27].

The code was altered further to suit the task such as using the command “results[0].boxes.cls” to list the IDs of all the objects detected [27]. A person has an ID of 0 and at the top of my script, I defined a variable called OBJECTS_TO_DETECT to be equal to 0 as a means of finding if a person was detected [27]. A for loop cycles through the list of items I want to detect (in this case just 1) before checking if the object has been found and setting a flag to true. This flag is used later to save the image of a person to a folder. Once ten images have been saved, the send email function is called (see the section Email Communication for more details) which then attaches images to the email before sending it.

```
# Run YOLO detection
results = model(image, imgsz=320, conf=0.8)
# Flag if any objects of interest are detected
detected_objects = results[0].boxes.cls.tolist()
object_found = False

for obj_id in OBJECTS_TO_DETECT:
    if obj_id in detected_objects:
        object_found = True
        print(f"Detected object with ID {obj_id}!")

# Draw bounding boxes on frame
annotated_frame = results[0].plot()

# Overlay FPS info
inference_time = results[0].speed['inference']
fps = 1000 / inference_time if inference_time > 0 else 0
text = f'FPS: {fps:.1f}'
font = cv2.FONT_HERSHEY_SIMPLEX
text_size = cv2.getTextSize(text, font, 1, 2)[0]
text_x = annotated_frame.shape[1] - text_size[0] - 10
text_y = text_size[1] + 10
cv2.putText(annotated_frame, text, (text_x, text_y), font, 1, (255, 255, 255), 2, cv2.LINE_AA)
```

Figure 6.23 Checking if an object was found & running inference

6.3.5 Testing YOLO in several scenarios

To understand how YOLO could be used for my project in the correct way, it was important to test it thoroughly in multiple use cases to grasp a full picture of what it can and cannot do. The same setup was carried out for each iteration of testing and only one aspect was changed for each test. The SaRQ was put on a platform, raised to the same height it would be from the ground if it was standing. Ahead of the camera, 2 metres were measured ahead and marked on the ground. A script was run that ran the SMTP server, the YOLO model and the MJPEG server. I laid down in the same location for each test in the same outfit for posterity. I was at the two-metre mark and laid down with my face visible to the camera for the first test. I would slowly move my hand to make it easier to identify that I was a person and continued until I was recognised ten times. In this case after the tenth time an email was sent to confirm that the model did in fact detect me where I could check that I was at the 2m mark and lying in the correct position. I repeated this test at a from a low confidence interval for the model or the percentage that the model was sure I was a human to a high one. I wanted to find out the maximum confidence interval for each test that I would pass. The next test saw me not wave which made it harder for the model to identify. I turned with my back to the camera and my face not visible and waved my arm for the third test and the fourth was with my back turned without any movement. Two tests consisted of me facing and turned away from the camera. In both instances I covered myself with pillows to block my torso and partially block my head to act as “debris” to mock a real scenario by making the scene more complex. I tested once during the night to see the drop in confidence for detecting me with my face to the camera and waving. It should be noted the interval was in the range of 0-1. The results were as follows:

The maximum confidence interval while my face was in frame:

With movement: 0.86

Without movement: 0.75

Covered in “debris”: 0.63

The maximum confidence interval while my face was turned from the camera:

With movement: 0.77

Without movement: 0.65

Covered in “debris”: 0.55

The night test:

With movement: 0.5

I learned several points from these tests. Mainly I confirmed that movement increased the chance of recognition, as did having a face visible for the model to recognise. The model could also identify me in complex scenes while on the ground with my back to the camera which was impressive even if at a low confidence. The main piece of interest I would like to note would be the fact that the model never misidentified me while testing. It would not recognise a person was in front of the camera if the confidence was too high for the given test, but there was no instance where another object was confused with a person or that I was mislabelled as another object in the dataset. A reason for this could be that the model was not in a low confidence interval until the debris testing. The night test resulted in a lower confidence score as anticipated, but it was still impressive that it detected me given how dark it was.

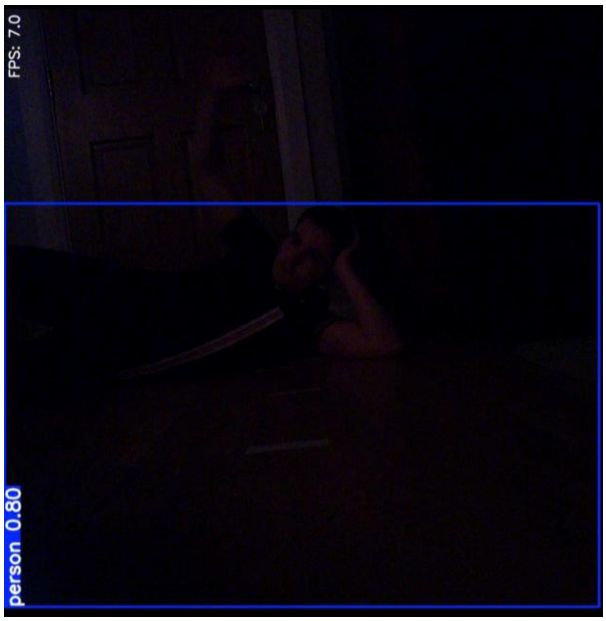


Figure 6.24 Night testing

6.4 Email Communication

Email communication was used for the SaRQ to alert SARP about potential humans detected by the YOLO algorithm. This was a feature used to avoid personnel having to accompany the SaRQ into dangerous areas and avoiding the personnel's attention to be focused on the MJPEG server constantly. With email detection, SARP could help another individual and upon getting an email can check the email attachments sent along with the alert, or view to the MJPEG server to see if a human was detected when they have time.

6.4.1 Why Email?

The reason for implementing email communication specifically was due to the already large scope of the project, paired with other factors. I knew I wanted to implement a method of alerting SARP of the fact that a human had been detected by the SaRQ but of course, in many disaster zones networks may not be stable. There were a few alternatives to email that I thought about such as sub gigahertz on the STM32IOT1A but that would require sending the images from the RPI somehow. If it was done over UART, the speed would be slow, and this is not accounting for the fact that a sub gigahertz module would be required to do this in the first place which would increase the cost of the project. The images could have been stored on the RPI and then when a secure connection was found, dumped but the issue of locating where the persons had been sighted would have been an issue. In the end, due to complexity and cost, I made the decision to use email.

6.4.2 Implementing an SMTP Server

SMTP stands for simple mail transfer protocol and is a standard for email transmission [29]. To send information from the SaRQ, I needed a sender and recipient email address as well as the details of a Gmail SMTP server such as the server, username, password, the port etc [29]. To create an account that could send emails over the server, I created an app password via the "security panel" when I opened the Gmail account [29]. I first needed to enable two step verification by selecting "Signing into Google" followed by "2-Step Verification" and "Get started" [29]. Once I completed the steps shown, I went to my Google account and searched for "App Passwords" [29]. I named my password "RPI" and created it. I saved the password and later implemented it in my script. Once I had a script that could send an email, there was an

issue I ran into whereby I could not send an image. This was an issue that required further research, leading to the eventual implementation of a function called send_email in the final code. If an object is detected this is where the image is stored on the RPI and the flag is incremented. The file is named example followed by the image number for the send_email code to located the images for attaching.

```
if object_found:
    if StreamingHandler.flag>= 0:
        StreamingHandler.flag+=1
        filename = f"/home/ferry/SaRQ-4th-Year-Project-/Python/yolo/detect scripts/img/example{StreamingHandler.flag}.jpg"
        cv2.imwrite(filename, annotated_frame)
```

Figure 6.25 The flag that calls send_email

If ten frames of a detected human are stored, the function is called to attach those to an email and then send it. There is a flag variable which is part of the StreamingHandler class (more information available in the MJPEG server section) which counts the number of frames saved before calling the send_email function once ten have been stored. The function works in a linear means where the sample code it edited to include a means of adding images to the body of the email.

```
if StreamingHandler.flag == 10:
    send_email()
    StreamingHandler.flag = -1
```

Figure 6.26 The flag that calls send_email

The sender, recipient and password are first defined, followed by the format of the email i.e. the subject sender and recipient. The body is attached to the email before the files are attached afterwards. A while loop iterates through the filesystem, opening the RPI's folder where the frames are stored before creating a MIMEBASE object named file, setting the payload and attaching it to the end of the object [30].

```
# set the subject and body of the email
msg['Subject'] = 'Test Email with Attachment'
msg['From'] = sender
msg['To'] = recipient

body = 'This is a test email with attachment.'

# attach the body of the email to the message object
msg.attach(MIMEText(body, 'plain'))
```

Figure 6.27 Formatting the email

```
while(count>0):
    # open the file in binary
    filename = f"example{count}.jpg"
    attachment = open(f"/home/ferry/SaRQ-4th-Year-Project-/Python/yolo/detect scripts/img/example{count}.jpg", "rb")

    # create a MIMEBase object and set its attributes
    file = MIMEBase('application', 'octet-stream')
    file.set_payload((attachment).read())
    encoders.encode_base64(file)
    file.add_header('Content-Disposition', "attachment; filename= %s" % filename)

    # attach the MIMEBase object to the message object
    msg.attach(file)
```

Figure 6.28 Attaching images to the email

MIMEBASE is a library used primarily for sending attachments with emails. A variable called count is compared for the while loop, counting down each time an image is attached until moving onto the next section of code. Once ten images are attached, the server is connected to via smtplib.SMTP('smtp.gmail.com', 587) [29]. 587 is the port specifying the server is using transport layer security and smtp.gmail.com is an argument specifying that the server is for Gmail [29]. TLS is then started using another command and server.login(sender, password) passes the sender and password (the app password created earlier) [29]. The email is then sent using server.sendmail(sender, recipient, msg.as_string()) specifying the sender, recipient and the message format before server.quit() is called to close the server.

```
# create a SMTP session
server = smtplib.SMTP('smtp.gmail.com', 587)

# start TLS for security
server.starttls()

# authenticate with the email account
server.login(sender, password)

# send the email
server.sendmail(sender, recipient, msg.as_string())
print("Email sent")

# terminate the SMTP session
server.quit()
```

Figure 6.29 Starting the server, sending the email and stopping it

6.5 MJPEG Server

A motion JPEG (or MJPEG) server is a web server with the ability to show live video on embedded devices [31]. I wanted to implement it due to the feature benefit of having SARP granted the ability to see the SaRQ's live view if a person is detected and the email attachments are blurry or obfuscated. Personnel can verify a person is visible if they are in frame lending a fallback to the email alert. There was also the issue of showing the live view of the camera via VNC viewer, where the RPI was having too much demanded on its system using YOLO and using the viewer resulted in too much ram usage and low frame rates. Once I implemented an MJPEG server, the stream led to an improvement in the quality of the video and a significant delay in latency with an increase in the resolution and fps becoming available.

6.5.1 Implementing an MJPEG Server

You must install the PiCamera2 library before running the server which is the command "sudo apt install -y python3-picamera2" [32]. The code is made of a page and three classes. The page is formatted to be simple html that is easily edited with a new title and other stylistic choices to suit your objective (see figure 6.30) . In the code, it is merely a title followed by the window size for the output to be streamed to.

```
PAGE = """\
<html>
<head>
<title>Picamera2 YOLO MJPEG Stream</title>
</head>
<body>
<h1>YOLO Object Detection Stream</h1>
<img src="stream.mjpg" width="640" height="480" />
</body>
</html>
"""
```

Figure 6.30 The webpage

The first class in the file is called StreamingOutput which serves to tell other threads when a new frame has appeared while also storing the new frame (see figure 6.31). It does this by creating a constructor, saving the latest frame as none and creating a condition object whereby it can notify other threads that a new frame exists [33]. In the write function, a frame is stored via the argument buf and condition.notify_all() tells other threads about this new frame [33].

```
class StreamingOutput(io.BufferedIOBase):
    def __init__(self):
        self.frame = None
        self.condition = Condition()

    def write(self, buf):
        with self.condition:
            self.frame = buf
            self.condition.notify_all()
```

Figure 6.31 The StreamingOutput class

The StreamingServer class is used to create a server which listens for requests and can create multiple threads for those requests (see figure 6.32). This is another simple class, accepting two arguments. ThreadingMinIn makes sure multiple threads are created i.e. each browser for the MJPEG server is a separate thread and server.HTTPServer creates the server which passes requests to the final class StreamingHandler [34][35]. allow_reuse_address = True is set so that the same port can be used immediately after disconnecting and daemon_threads= True is used

so that threads created can be terminated and the server won't have to wait until they are finished to stop [34].

```
class StreamingServer(socketserver.ThreadingMixIn, server.HTTPServer):
    allow_reuse_address = True
    daemon_threads = True
```

Figure 6.32 The StreamingServer class

The final class is called StreamingHandler. This contains the most code, processing requests and showing the stream in a browser.

Breaking it down further, the first place to look is the camera being initialised before a variable named output is set equal to StreamingOutput. The camera begins to record, setting the format to JPEG with JpegEncoder. FileOutPut(Output) is telling the camera to put the frames to the StreamingOutput class which takes new frames and alerts all threads that a new frame exists before saving it [36]. The server is started in a try-catch where the address is set to port 7123 and server is set to a StreamingServer object where the address and the StreamingHandler are two arguments passed. The server is then kept alive forever using server.serve_forever [34]. A finally keyword is used to stop the camera recording cleanly if any error would occur.

```
#Initialise camera
picam2 = Picamera2()
picam2.configure(picam2.create_video_configuration(main={"size": (1280, 1280)}))
output = StreamingOutput()
picam2.start_recording(JpegEncoder(), FileOutput(output))

#Start server
try:
    address = ('', 7123)
    server = StreamingServer(address, StreamingHandler)
    print("Server started")
    server.serve_forever()
finally:
    picam2.stop_recording()
```

Figure 6.33 Code showing how the Pi camera is turned on and the MJPEG server is started

So, in essence StreamingOutput is used to tell other threads or browsers when a new frame arrives, StreamingServer is used to accept HTTP requests and send them to the StreamingHandler, but how does StreamingHandler work?

Well, StreamingHandler consists of a function called do_GET to handle get requests, where information is asked via the client or browser via the argument server.BaseHTTPRequestHandler [35]. There are three paths that the browser can request. "/" returns a 301 or moved permanently error before sending the browser to "index.html" [37].

If the path is "index.html" the response is 200 for ok and content is a variable that is set to the Page. The Page as explained is merely a title and an img tag that directs to the MJPEG server via src="stream.mjpg". Content has its type set to "index/html" before the length is calculated and self.wfile.write(content) is used to send the webpage byte by byte to the browser [35].

The final path option is "stream.mjpeg" which sends the response 200 before telling the browser that the information will be sent in pieces [38]. The browser basically makes a request to see the index once. The index has an img tag which asks for the MJPEG server to be sent and is looped for each frame. There is an exception in case of errors and outside of the if statement checking the "/stream.mjpg" path and the others, there is an else in case an incorrect path is requested.

```
elif self.path == '/stream.mjpg':
    self.send_response(200)
    self.send_header('Age', 0)
    self.send_header('Cache-Control', 'no-cache, private')
    self.send_header('Pragma', 'no-cache')
    self.send_header('Content-Type', 'multipart/x-mixed-replace; boundary=FRAME')
    self.end_headers()
```

Figure 6.34 The stream being sent part by part

A while True is combined with the other functions from previous sections as a main loop for the SaRQ on the RPI. A check is performed if the serial buffer has been sent anything. The SaRQ sends responses after each command has been performed (such as a step forward) and the RPI upon receiving a response would call the serial function. This is the same logic from the Serial

Communication section, edited to perform collision detection logic (see Collision Detection for more details). After the check, the code performs analysis using the YOLO model code (see YOLO implementation for more detail) to analyse the frame. This annotated frame is then sent to the client to end the while True loop.

```
# Encode frame back to JPEG for streaming
ret, jpeg = cv2.imencode('.jpg', annotated_frame)
if not ret:
    continue
annotated_bytes = jpeg.tobytes()

# Send MJPEG frame to client
self.wfile.write(b'--FRAME\r\n')
self.send_header('Content-Type', 'image/jpeg')
self.send_header('Content-Length', len(annotated_bytes))
self.end_headers()
self.wfile.write(annotated_bytes)
self.wfile.write(b'\r\n')
```

Figure 6.35 Each frame being sent to the client

6.6 Collision Detection

Collision detection was a requirement for the project for the SaRQ to navigate tough terrain in disaster situations. It needs to be able to detect anything that it could run into and avoid damaging itself or any of the hardware mounted to it.

6.6.1 Why the HC-SR04?

Collision detection for the SaRQ consists of a HC-SR04 ultrasonic sensor attached to an SG90 servo motor. This was chosen as the method of detection over implementing a 2D LiDAR which would have increased the scope of the project for numerous reasons. Firstly, a 2D LiDAR would be able to attach to the RPI as a hat, but a solution such as an extended mount for the RPI would be required to lift it above the wiring in the front and rear of the SaRQ to scan the surroundings (see the section on building the SaRQ to learn more about the wire management issues). This mount would have had to not only fixed the RPI higher but be rigid enough to not fall off of the SaRQ and would have had to keep the centre of gravity in the same spot for the

SaRQ to move correctly. This would be before programming begins. The 2D LiDAR would have had to be programmed which would have taken longer than the HC-SR04 but in the end would have provided better collision detection. A 2D LiDAR works similarly to the HC-SR04 where both send out a signal and receive one back to calculate the distance to an object, it is just that the LiDAR uses a pulse of light but the ultrasonic uses sound [39][40]. The main difference is that the ultrasonic can only read directly ahead, but the 2D LiDAR can check in a 360-degree view leading to a better field of view [39][40]. Due to the hardware and software scope increases, the HC-SR04 was chosen. Another reason I chose the HC-SR04 over the LiDAR was my previous experience programming it. The HC-SR04 was attached to a servo motor for rotation with a customised 3D printed component to secure the HC-SR04 to the servo motor (see Building the SaRQ for information). This mount did not require the RPI to be raised and by being attached to the chassis was sturdier than raising a platform from the centre of the chassis.

6.6.2 Theory of Operation

Similar to bats using high frequencies for echo location, the ultrasonic sensor uses two ultrasonic transducers to send and receive 40kHz pulses of sound [40]. The sensor measures in the range of 2 centimetres (cm) to 4 meters (m) with an error range of +-3 mm [40]. The receiver creates an electrical pulse proportional to how far away an object is and this is how the distance is calculated [40]. The HC-SR04 works at 5 volts, but given that the RPI is a 3V3 device, a voltage divider circuit is used to return 3V3 back to the RPI so as not to damage it (see Building the SaRQ for more details) [40].

There are four pins, VCC (this stands for voltage common collector which is a positive supply), GND (the ground signal), Trig (Trigger) and Echo [40]. VCC is connected to the RPI for 5V and GND is connected to ground respectively [40]. The Echo pin is the signal output pin while Trig is the signal input pin [40]. If this pin is pulled at high voltage for 10 microseconds (µs), the sensor sends out 8 pulses at 40 kHz through the transmitter [40]. The pulse helps the receiver identify that the signal is coming from the transmitter and isn't background noise [40]. Once the pulses are sent out, the Echo pin is held high until the signal bounces off an object and returns [40]. The Echo pin goes low upon receiving the signal that has bounced off an object and depending on the length of the Echo pulse, the distance between the sensor and the object ahead is found

[40]. If no wave bounces back, the sensor halts waiting after 38ms. This timeout means nothing is in the detection range of 4m [40].

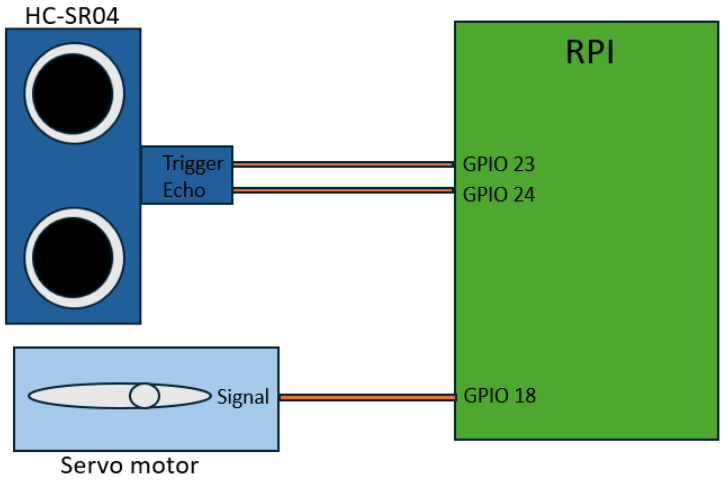


Figure 6.36 RPI, HC-SR04 and servo motor connections

Distance = Speed / Time is used to calculate the distance to an object [40]. If the Echo pulse is 500 µs and given that the speed of sound in air is 0.0034 m/µs, you can plug in that D = 0.0034/500 to give a distance of 0.085m or 8.5cm [40]. This is only half the story though since the wave travels to an object and the way back, this means that the final value must be halved. In this case it would result in 4.35cm.

6.6.3 Programming Collision Detection

The code for collision detection involves two segments. The first is for the HC-SR04 to get the distance ahead. The second is to get the servo motor controlling it to rotate. The loop for the SaRQ works so that every movement the STM32 completes, it sends a movement complete message to the RPI. The RPI checks if it has received anything and takes a reading from the ultrasonic sensor in the serial function. If it reads a distance less than 15cm, a servo function is called.

```
def serial():
    if ser.in_waiting > 0:
        received = ser.readline().decode('utf-8').strip()
        print(f"Received: {received}")
        # Check distance & Send message
        read_distance = ultrasonic()
        if read_distance < 15:
            direction = servo()
            #Check if you are turning right
            if direction:
                ser.write(b"4")
                print("Sent: 4")
            else:
                ser.write(b"3")
                print("Sent: 3")
        else:
            ser.write(b"1")
            print("Sent: 1")
```

Figure 6.37 The serial function

The servo function rotates the ultrasonic sensor to the left, calls the ultrasonic function and stores the result in the left_dist variable. The same logic is performed for the right side, and the servo resets the ultrasonic to the centre before comparing the left and right distance. If the right distance is greater, the SaRQ turns right, otherwise, the SarQ turns left (this is done to fix potential issues where the measured distances are equal). Based upon the chosen direction, the RPI sends the number for left (3) or right (4). The two values are read and the SaRQ decides which side it will turn to avoid a collision.

```
def servo():
    #Left
    pi.set_servo_pulsewidth(18, 500)
    left_dist = ultrasonic()
    print(f"Left is {left_dist}")
    sleep(1)
    #Right
    pi.set_servo_pulsewidth(18, 2500)
    right_dist = ultrasonic()
    print(f"Right is {left_dist}")
    sleep(1)

    #Reset to center
    pi.set_servo_pulsewidth(18, 1500)
    sleep(1)

    #decide which direction to turn
    if right_dist > left_dist:
        return 1
    else:
        return 0
```

Figure 6.38 The servo function

I began with programming the servo motor and found code that would work but introduced an issue where the servo motor would not stay in place but rather wiggle at an angle. An example would be setting it to 10 degrees, only for the servo to go from 15 degrees to 8 degrees non periodically. I had this issue before when I didn't connect the servo motors, I was programming via the STM32 to a common ground. The addition of a common ground did not fix the issue here. I had a power supply and changed the voltage range but given that the same range was used for the STM32 servo and worked, I did not see it working. For context I used the same servo motor model for the RPI as I did with the STM32. The servo motor still wiggled at varying voltage ranges. I looked up a jitter free means of programming a servo motor for the RPI and found a source that worked immediately. This source used the command "sudo pigpiod" before using a script that imported the pigpio library for control and time for delays [41]. An object was created in the example named pi via pi = pigpio.pi(). Using commands such as pi.set_servo_pulsewidth(18, 1500) I could rotate the servo to 90 degrees [41]. I used trial and error to find the 0 degrees, 90 degrees and 180 degrees pulse widths.

The ultrasonic sensor code was based off a tutorial using the RPI.GPIO library to control the general-purpose input/output (GPIO) pins on the RPI for the purposes of using the HC-SR04 [42][43]. The code is a written version of the theory of operation where the trigger and echo pins are defined and setup as input and outputs before the pulse is sent out via the trigger pin and the length of the pulse that the echo pin has is used to determine the distance [43]. The distance is the pulse length for the echo pin, divided by the half of the speed of sound in air converted into cm per second instead of meters. $343ms^{-1}$ is the speed of sound in air which becomes $343000cms^{-1}$ and halved becomes $17150cms^{-1}$ which is plugged into the formula [43][44]. The reason for halving the speed is that the sound wave travels to the object and back to the ultrasonic sensor [43]. This means the distance measured is double unless you multiply the result by ½ which is what is happening here [43][44].

```
def ultrasonic():
    GPIO.setup(TRIG, GPIO.OUT)
    GPIO.setup(ECHO,GPIO.IN)
    GPIO.output(TRIG, False)
    time.sleep(2)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)
    while GPIO.input (ECHO)==0:
        pulse_start = time.time()
    while GPIO.input (ECHO)==1:
        pulse_end = time.time()
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150
    distance = round(distance, 2)
    print(f"distance is {distance}")
    return distance
```

Figure 6.39 The Ultrasonic function

7 Part III: The Mechanics

7.1 The Mechanics within the SaRQ

The SaRQ did not just involve problem solving on the hardware and software sides, but also mechanically. It involved debugging and reiterating mechanically alongside the hardware and software. The SaRQ was full of testing due to the nature of building the project whereby the servo motors had to be calibrated, and offsets calculated so that the SaRQ could not only balance but move forward repeatably. The mechanical segment of the project involved three sub-sections; the design, the build and finally testing the balance of the SaRQ via a plethora of avenues.

7.2 Designing the SaRQ

The design of both the legs and the chassis were the result of the cumulative work in researching designs that already existed and testing to find any fault or flaw with the designs. The components were 3D printed and went through multiple iterations.

7.2.1 The Legs

Designing the legs was a multi-step process. I started by researching other designs. This was the correct choice as while I wasn't using a pre-built kit, it made sense to see what existed already and analyse that before moving ahead with my own design. I watched three videos, each with their own designs to break down. The first video showed the creation of a turtle quadruped that dragged itself along the floor rather than walked in balanced steps [45]. I dissected this design and broke it down below.

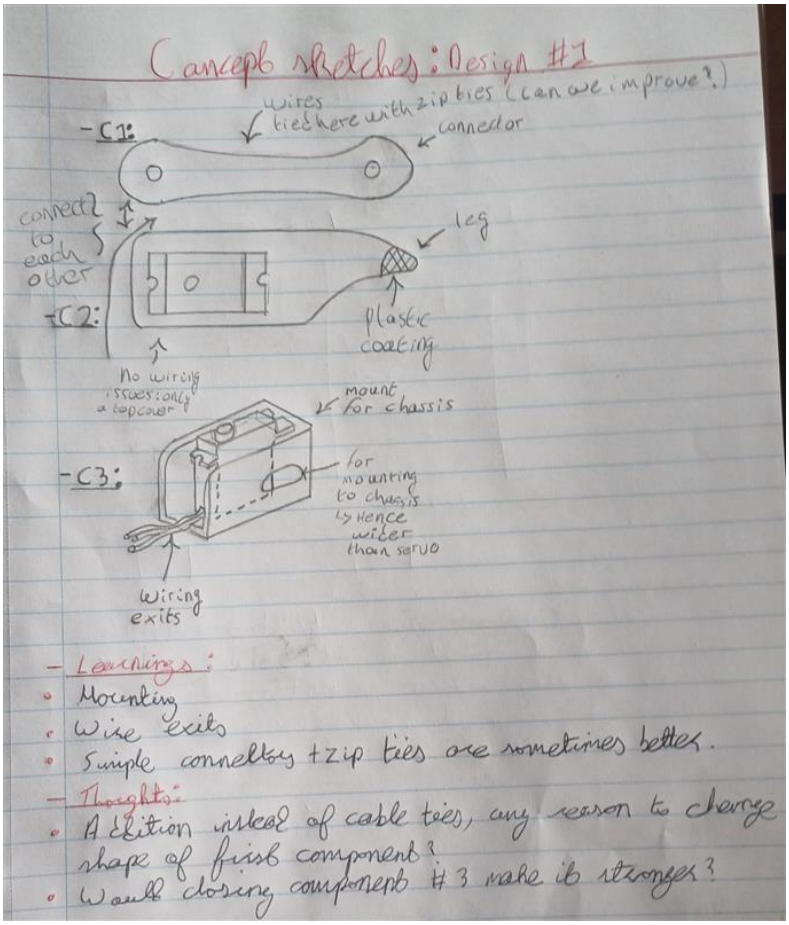


Figure 7.1 Breaking down leg design 1

It was important to understand what each design taught me and how I would improve upon it, given the chance. The legs consisted of three components, a connector, a leg and a mount. It was during this breakdown that I figured out how the leg connected and how each axis was moved by a specific servo. The connector seemed quite efficient in terms of plastic use, but also weak in the midsection. It formed the upper leg, and I thought that with the addition of a groove along it, cable management would improve. The wires could slot in and be held via zip ties or other bonds. The leg or C2 seemed efficient in design. It had enough of a wall outside of the slot for the servo and even a plastic coating for the tip of the leg to add more grip. The mount was the last piece of the puzzle and was confusing. It could have been more efficiently designed. For example, the front wall was open when closing it would be more rigid. The lack of a press fit for the servo housed within also felt inefficient.

Having dissected one design, I moved onto another. The second was similar to the first, but unlike the small leg on the first design, the second favoured a smaller upper leg (see figure 7.2).

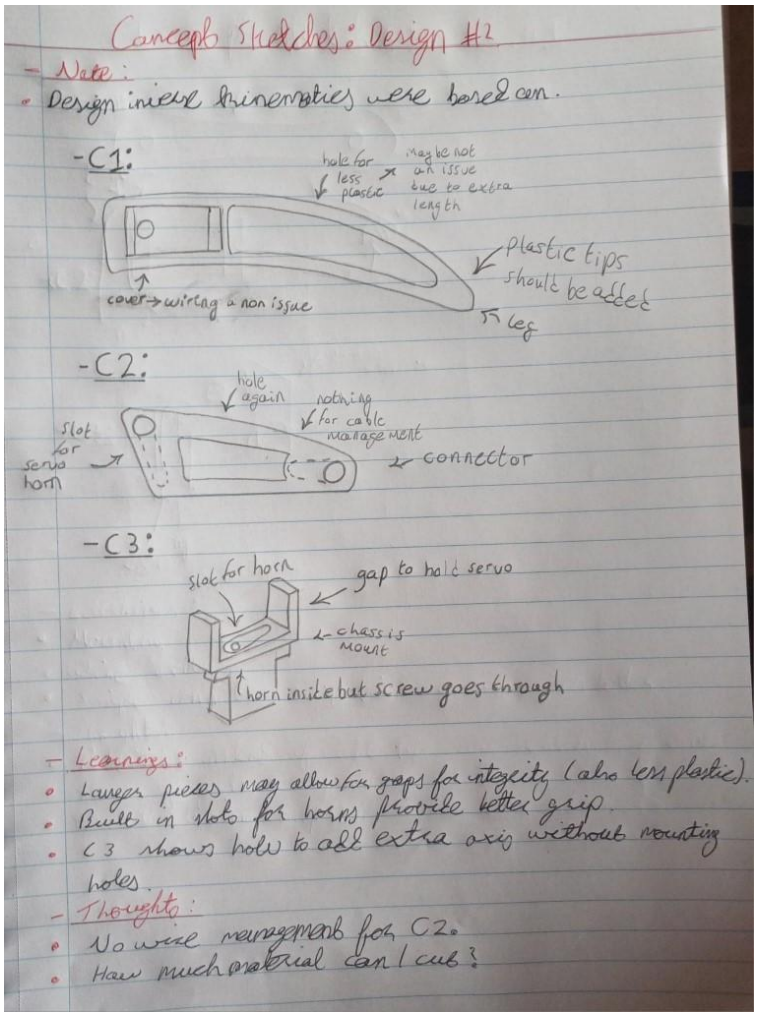


Figure 7.2 Breaking down leg design 2

This design was intended for a six-legged robot and appeared to be stronger and more efficient in design [15]. It too consisted of three segments, but whereas design 1 had a short leg and weak connector, this design appeared more rigid. The leg did not have plastic tips, but I knew I could add it to my design if I chose. I did not want to put on anything for added grip in case friction worked against it and the robot had difficulty walking. I liked the leg much better as I knew I wanted my robot to have some height off the ground. I did not want it to walk along the floor but above. This leg had a press fit for the servo motor as well, but with the space in the middle and walls around it felt like an improvement on the leg in the first design. The connector

of this design follows on from the leg in that the walls are on the outside with space within to save on plastic without harming the structural soundness of the design. The pressure is exerted not directly on the middle as in the first design but given two paths to move through which is a bonus in my opinion. It also had two servos connected to it and covered the tops which added structure but would later cause me to make an error in my own design as you will see later. The final part or C3 seemed much cleaner in terms of servo connection and the width could be made to a press fit for the servo that would be attached.

With all of this in mind, the main inspiration for my design came from the second design. The structural integrity seemed more stable, and it suited my plan of having a design higher off the ground. I did intend on adding grooves into my design for wiring but decided against it to focus on more important mechanical issues such as motion and balance. If I wanted to incorporate wire management into the design, the design would need to be almost finished to warrant it.

It was up to me to design the legs at this point, and I used Onshape, a tool I had experience with and had used on multiple previous projects for 3D printing. I started by drawing the first versions of the designs, measuring the sizes of the servo horns and widths of the servos for the designs with vernier callipers for accurate measurements.

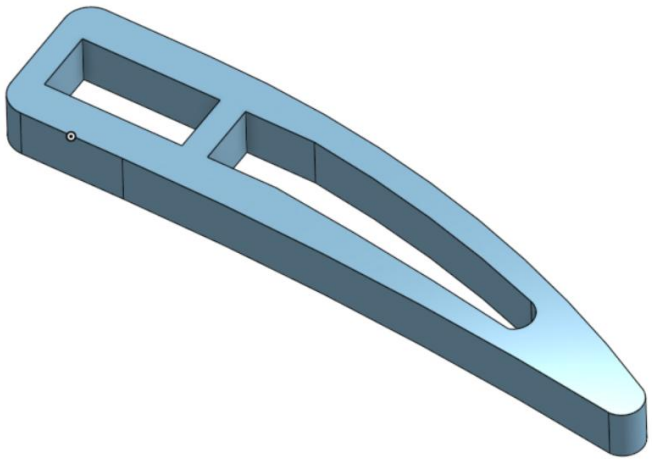


Figure 7.3 The lower leg design

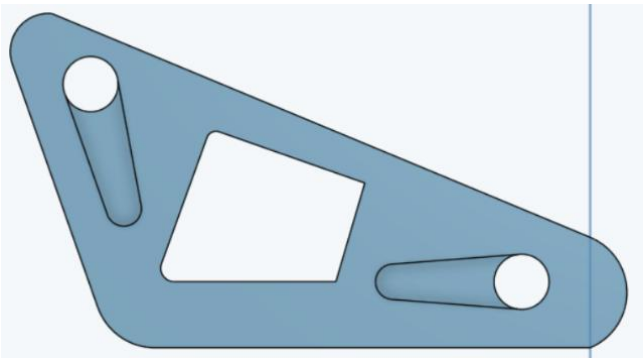


Figure 7.4 The first connector design

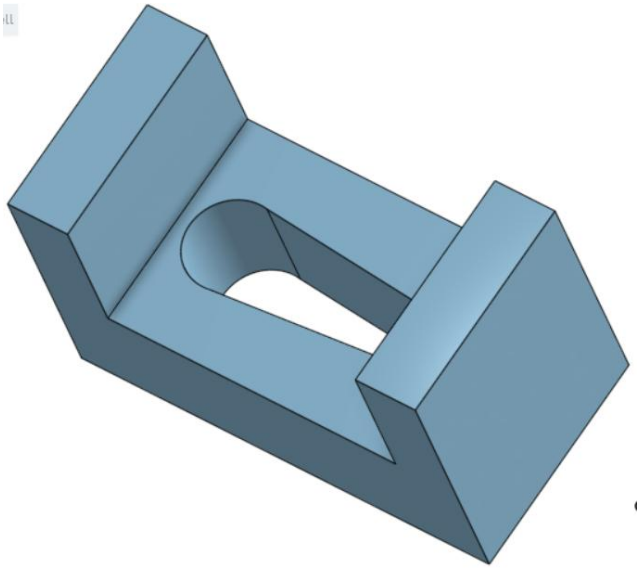


Figure 7.5 The hip connector design

Once drawn, the parts were printed and the first leg was assembled. I used a thickness of 1 centimetre for the width of the connecter, the leg and hip component (C3 in the previous diagrams). This was done for components that would be light but still strong. The prints were done in such a way that each component had three walls internally to make sure less plastic was used to be both efficient and save on weight. At the start, the servos were attached via a press fit and blu tack (see figure 7.6).

Figure 7.6 The prototype, fastened with blu tack

It is evident that the leg is leaning due to the blu tack not providing a hold for the servo horn. This was an issue, along with the fact that the servo horns were not screwed in place during this testing period. During testing, only one leg was printed and controlled to see if there were any unforeseen problems that arose. This was done as servo calibration was still being figured out and the direction of motion was changing due to my work on inverse kinematics (touched upon earlier in the report). The legs were secured without blu tack further on in the project timeline. I had to redo the connector prints due to the inverse kinematics I was developing. The problem came from the fact that the servo could only connect from one side. The IK model meant that servo motors had to rotate in specific directions and by only connecting from one side my legs wouldn't work with the calculations. This was rectified in the second iteration of the connector print.

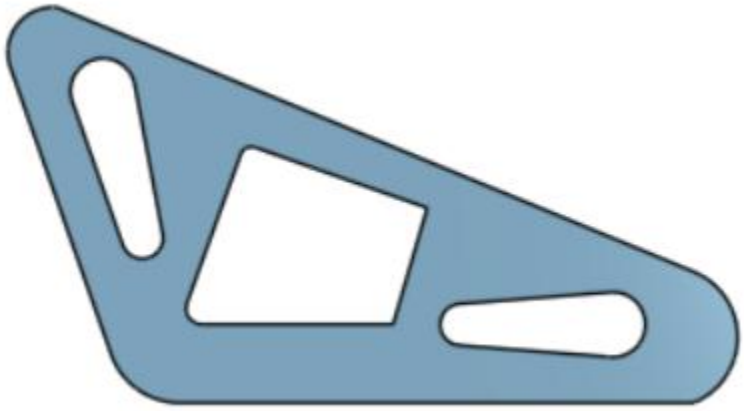


Figure 7.7 The second connector design with a through hole cut

The second iteration of the connector allows the servo horns to connect from either side. This was the last change undertaken for the leg designs. Three more legs were printed after this with the same designs.

7.2.2 The Chassis

Designing the chassis was error free. At first, the plan was to have the STM32IOT1A and RPI sitting in front of one another, but this was later changed. It was a large print, including width for the STM32IOT1A and RPI along the centre based on the widths and lengths taken for both incorporated into the drawing on Onshape. The width for the press fit for the servos to slot into the leg was taken for this design, used to allow the servo motors that would control hip movements to sit in the chassis at the four corners.

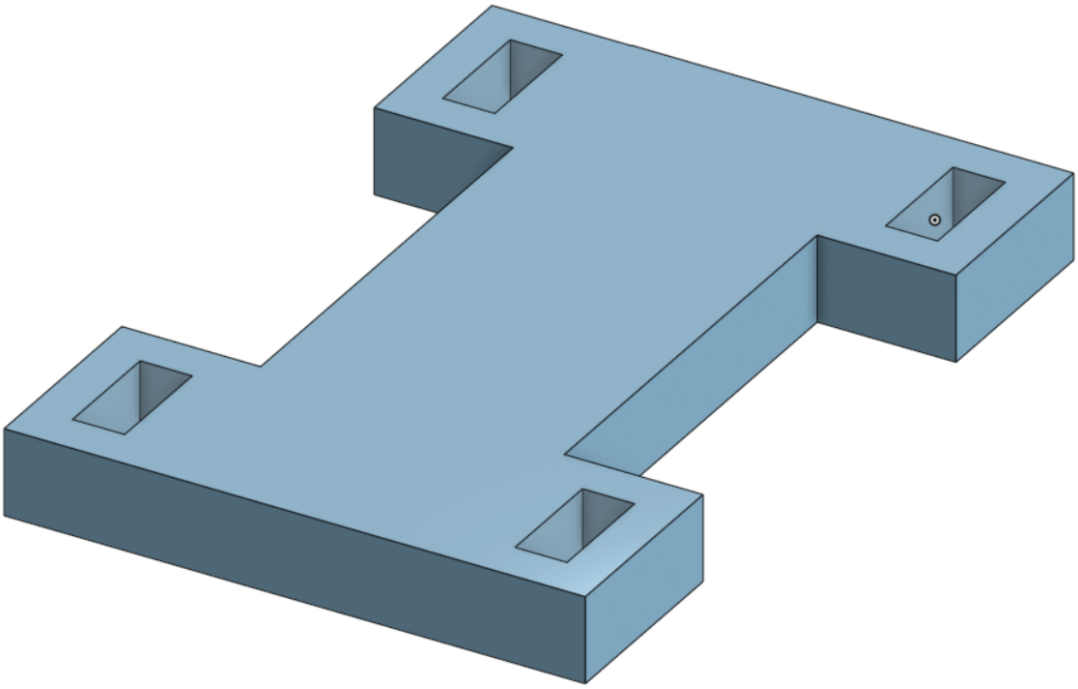


Figure 7.8 The chassis design

Once I added the servo motors to the design, no other issues arose. This is because I used forethought to the design due to the issue with the connector. It looks simple, and that is part of the design. With the corners being the only extensions from the main body, the legs can stand and rotate anywhere. The front legs can lean with the tips touching in front and the rear legs can either mimic this or fit the opposite way to give further support. A good design allows you to have options, which is why I am happy with this (see figure 7.9 and figure 7.10).

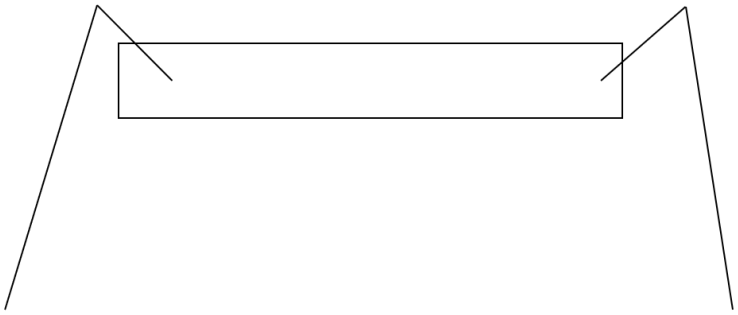


Figure 7.9 The chassis with legs attached facing opposite

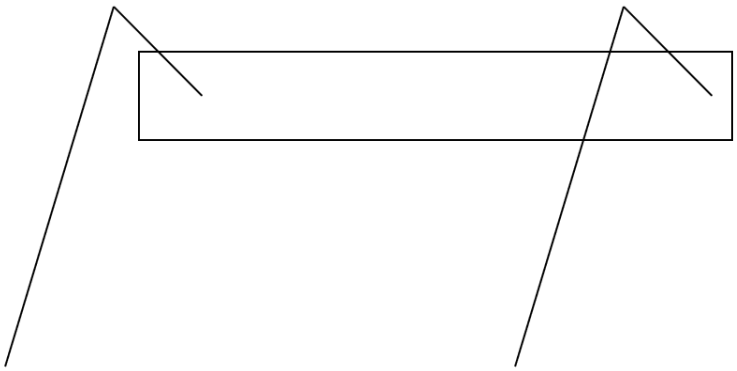


Figure 7.10 The chassis with legs attached facing the same direction

7.2.3 The HC-SR04 Holder

In the final stages of the design process, I realised that I would need a mount for the SaRQ to detect collisions. The solution involved designing a means of holding the HC-SR04 ultrasonic sensor in place, while providing a means to mount the holder to a servo motor. The design I came up with held the ultrasonic sensor via the receiver and transmitter, while having a base to drill through for mounting (see figure 7.11).

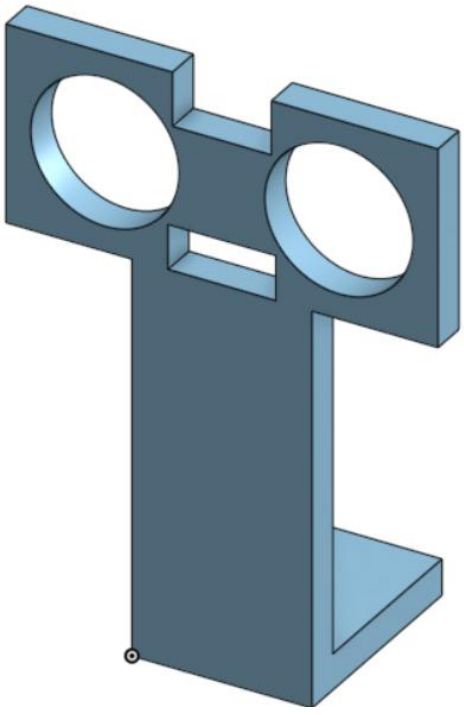


Figure 7.11 The ultrasonic mount

7.3 Building the SaRQ

The SaRQ began as a single leg paired with an STM32IOT1A and RPI during the testing stages, only held together by the press fit of servo motors and blu tack. After finalising the leg design and creating the chassis design a new task emerged. I had to print and build the entire SaRQ in a way that wouldn't damage any of the plastic and yet provide stability to allow the SaRQ to walk. There were three problems that emerged throughout the build process. I had to secure the legs and chassis together, create a strip board and the issue of making a mount for the PI cam and the HC-SR04 to be able to detect humans and collisions respectively.

7.3.1 Securing the Chassis and Legs

The prints have three walls of plastic to make them light yet strong, but this left a risk when building the SaRQ. It meant that when any hole was drilled for the purpose of inserting a screw, there was the risk of cracking a component. When I used a drill, I had to be slow and methodical to avoid an accident occurring. To assemble the legs, they were calibrated to move according to my IK model (see the section on inverse kinematics for more detail). Initially, two legs could attach to the connector with respective angles above and below. The servo motors for the legs on the other side needed to be able to attach on the other side of the connector for the IK model to work. This is the reason for the connector redesign. For example, for the shoulder servo motor, the angle on top needed to be 0 degrees and 180 degrees for the bottom with the servo turning in the correct direction. Unless the servo motor could be attached from both sides this wasn't possible as the servo motors only rotated in one direction. With four legs created, I had to find a more suitable means to make them rigid than simply using blu tack. I thought of the components rather than the leg as a whole in terms of securing, as it was easy to see where problems could emerge if you narrowed your focus. With further testing, issues regarding the whole leg would come out and further adaptations could be made if required. Before starting on drilling, I inserted all of the hips into the chassis and found that like the leg components, there was a clean press fit that did not require any screws to secure them in place. There was no press fit for the shoulder servo held in the hip component, and it did end up requiring a screw to secure it for this reason. Any other screws were mainly aimed at securing the servo horns themselves.

Beginning with the connector, the end of the servo horn attaching to the lower leg was secured via a screw inserted perpendicularly. This was seen as the best place to put it as the base of the horn is attached to the connector via this screw. It should be noted that every time I drilled a hole for a screw, I would use a tapper to tap grooves into the hole for the screw's thread to grip to. This added another step to an already long process but was worth it. This made each screw hold firmer and helped immensely with structural integrity. A similar hole was created for the other servo horn to be held in place as well.

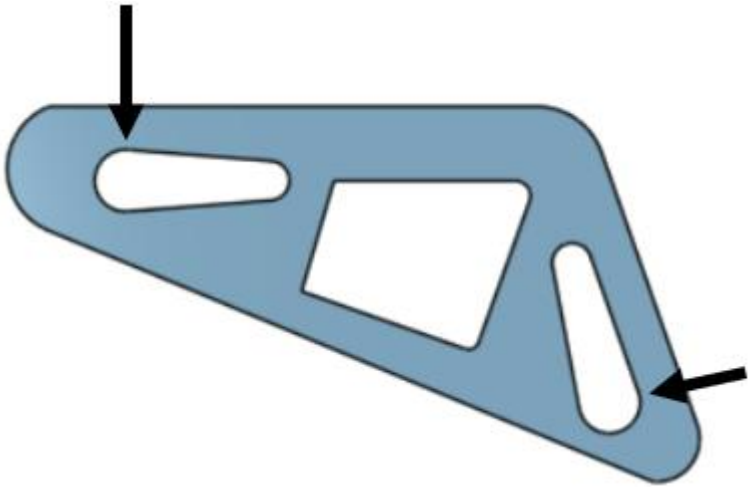


Figure 7.12 The connector with arrows indicating screw locations

The hip component was intriguing as I knew I had to be more delicate with it. There were two screws going into this part. On one side, a screw to secure the servo that slot into the component was held in place. The screw is a third of the way up the part as if it was further up the risk of cracking rose. The second screw was inserted into the opposite side to avoid weakening the structure too much. The risk of cracking with two holes on one side of the component was too high. This second screw held the base of the servo horn like the connector screws.

Figure 7.13 The hip with arrows indicating screw locations

The screws were far too long initially, sticking out and needed to be shortened. I had a bag of M3 screws that I used when first building the SaRQ, but it was obvious that they made the build look more amateurish and more importantly they interfered with the range of motion of the SaRQ as the screws would hit into the wiring. The SaRQ was tested after all these screws were inserted and the balance was not to my expectations. I tried getting the SaRQ to walk but no matter what, it would collapse. This is where I made two improvements. I inserted a new screw on the upper segment of the servo horns of the connector. The main issue at hand was that the servo horn was held at the base, but the top of the horn would rotate (see figure 7.14).

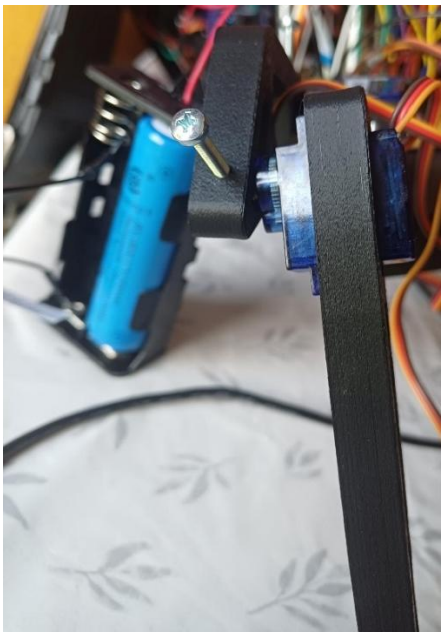


Figure 7.14 The unsecure servo horn causing the leg to tilt

To address this and help prevent the legs collapsing, I inserted two new screws into each connector after drilling and tapping (see figure 7.15). This made the legs much sturdier, and I was finally able to begin rough walking animations. While I was doing this, I had ordered shorter screws and they finally arrived. I changed each screw in the SaRQ, and it looked a lot cleaner and the issue with screws getting caught in the wiring was no longer an issue.

Figure 7.15 Shortened screws securing the leg

7.3.2 Creating a Strip Board

The strip board was born out of a necessity as wire management was an issue that needed resolving. The first time I connected everything to the SaRQ, including the servo motors, RPI and STM32IOT1A it looked like a nest (see figure 7.16). I labelled the signals to make it easier to trace everything, but I knew it needed to be changed if the SaRQ was to work.

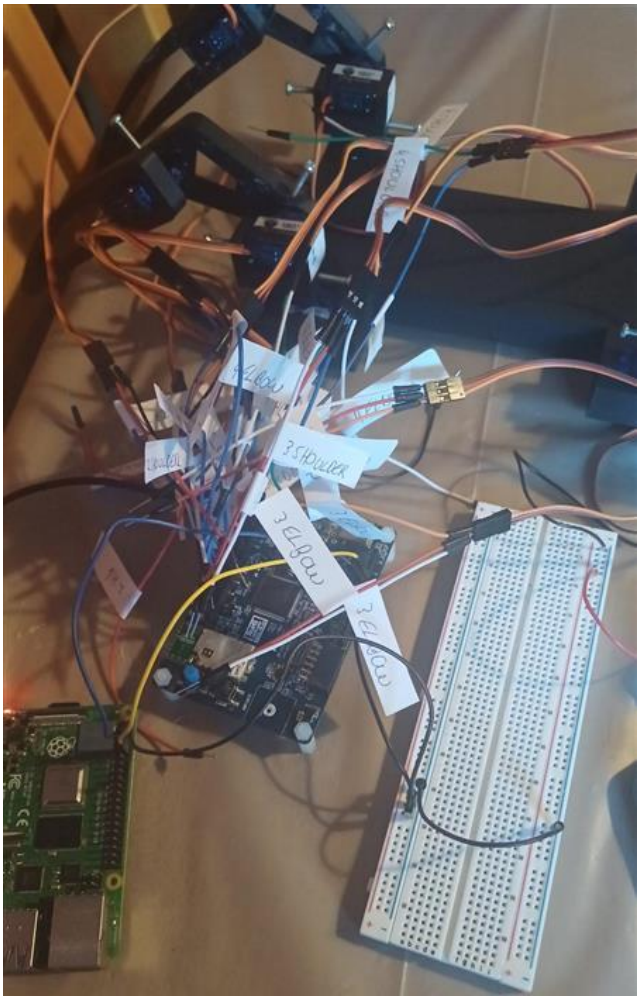


Figure 7.16 A nest of wiring

This was due to wires being connected to the servo motors mainly. They were far too long and since there were signal, power and voltage lines that all split, they couldn't be tied together. I created the first design for the strip board with the idea to have each servo motor connect to either the front or the back of it in a row of headers. The voltage rails would connect to the power via the board along with grounds from the servos. The signals would have a pin in front of them to connect to the STM32IOT1A as well as a pin in line with the other power and ground headers. I cut the wires to length and soldered everything together to create the first iteration (see figure 7.17).

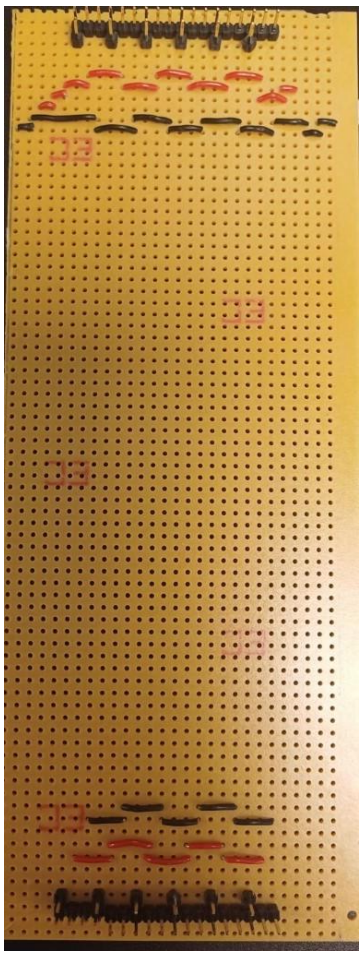


Figure 7.17 An earlier version of the stripboard

I made tweaks as I went on. Initially, I found the datasheet for the strip board to understand how much current could flow per rail and I realised than I had to spread it across five rails either side to be safe. I had the rails connected in series initially which would carry an exorbitant amount of current. Those rails would not be able to handle the cumulative current without burning up. I moved the headers closer now that I knew the size of my circuit which required a lot of desoldering and resoldering (see figure 7.18).

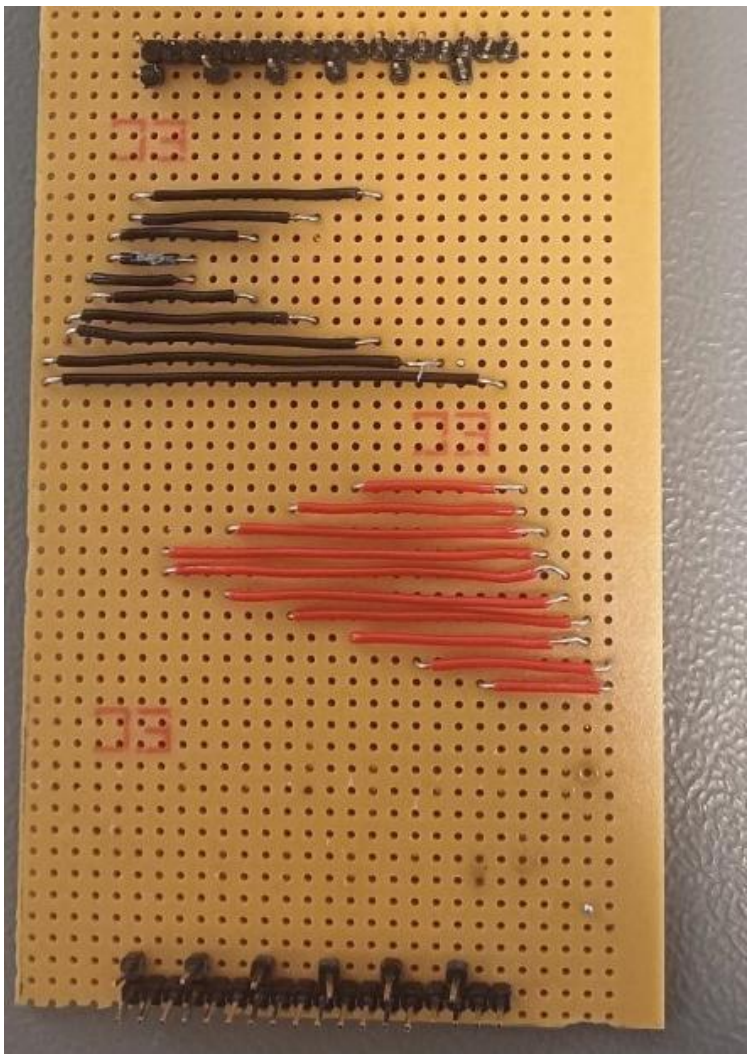


Figure 7.18 The strip board with closer headers

I realised when I began to work on collision detection that the HC-SR04 required a potential divider circuit to operate. This was since the RPI can output 5 volts but is itself a 3V3 device. The potential divider acted as a means of letting 3V3 return to the RPI to complete the circuit without damaging it (see figure 7.19).

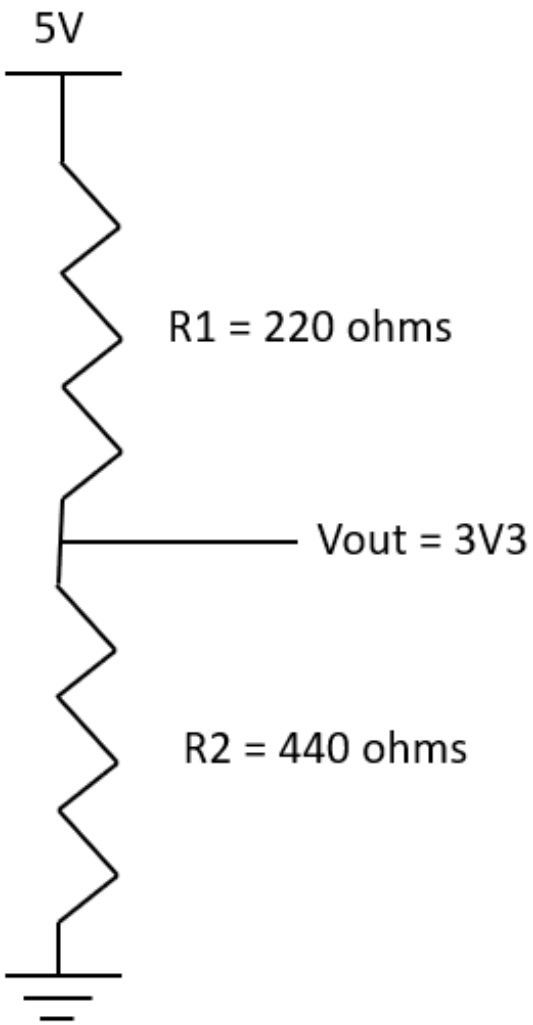


Figure 7.19 The potential divider circuit

The formula for a potential divider is as follows:

$V_{out} = (V_{in}R_2)/(R_1 + R_2)$ where V_{out} is the output voltage, V_{in} is the input voltage and R_1 and R_2 are the resistors respectively.

Potential dividers work as ratios of voltage, so if I wanted 5 volts output from 10 volts, R1 would need to be the same as R2. If R1 was 100 ohms or 1000 ohms it doesn't really matter (unless you make it so large of a resistance that no voltage can push current through). Since I am inputting 5 volts and want 3V3 as an output voltage, I used the formula:

$3.3 = (5R_2)/(R_1 + R_2)$

$3.3(R_1 + R_2) = 5R_2$

$3.3R_1/R_2\ 3.3 = 5$

$3.3R_1 = 1.7R_2$

$R_1 = 0.51R_2$

In this case, R_1 was twice R_2 as a ratio. I added the resistors required which in this case were 220 ohms for R_1 and 440 for R_2 before adding new headers for the HC-SR04 and servo motors to connect their voltage and ground pins to. Note that R_2 is two 220-ohm resistors which are in series and acts as a single 440-ohm resistor in the divider. I soldered a common ground for the STM32IOT1A and RPI next. At the top of figure 7.20, there are two wires as I conducted continuity testing before using the board and noticed that two rails were not connected. This was due to the rails falling off while I was soldering. I tried soldering over the piece that fell off but this did not fix the issue and there was no continuity. I added the wires to connect everything and more headers at the top so the power source could connect to the board.

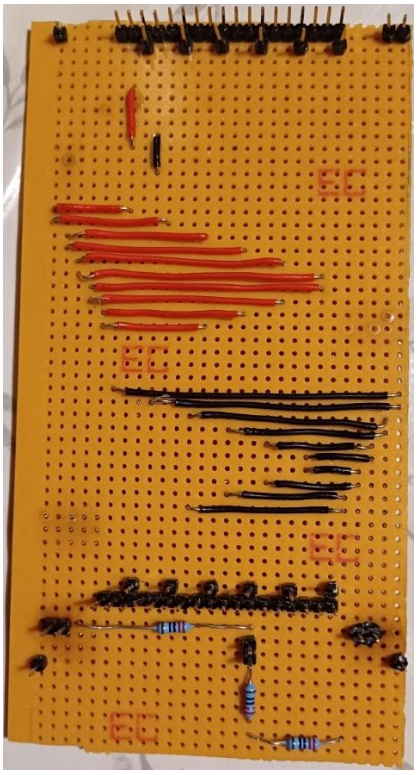


Figure 7.20 The final iteration of the stripboard

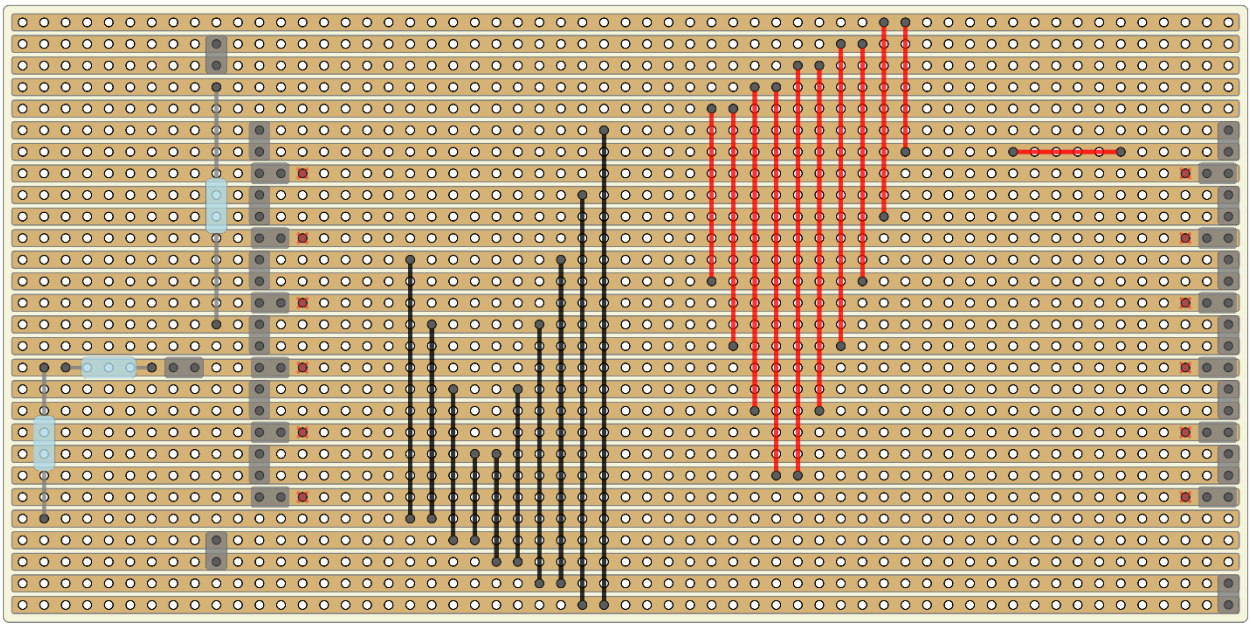


Figure 7.21 The final iteration of the stripboard in schematic format

Note that the red Xs on the schematic above show where I broke the rails so that the PWM signals on one side of the board did not connect with their opposite. Connecting the stripboard also proved a last tweak was required for the wiring to be clean. The problem was the initial thought process of putting the STM32IOT1A in front of the RPI along the chassis took up a lot of space and made the legs restricted in movement. The connections for the servo motors to the STM32IOT1A on one side would be closer than the other side which would create a tangled mess of wires. I made the strip board in the first place to avoid making wires too long. I added new legs to the STM32IOT1A to make it sit above the RPI. This solved the problem of servo motors being too far away from the STM32IOT1A pins and restricting the movement of the legs. The stripboard was cut to size and slot underneath the RPI (see figure 7.22). Rubber bands were used to wrap bundles of wires together such as the servo motor connections at the front and rear of the SaRQ.

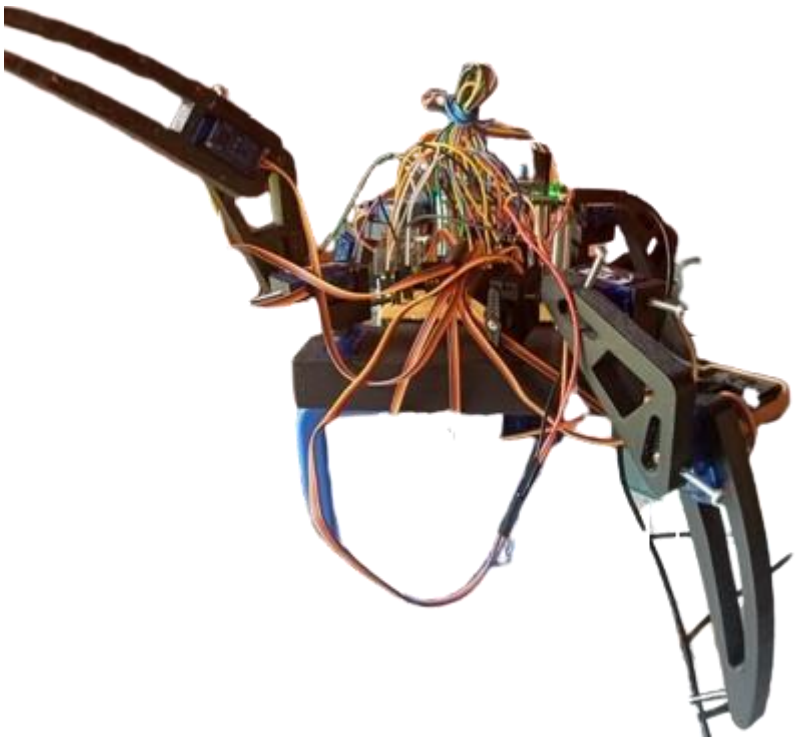


Figure 7.22 The SaRQ with better wire management

7.3.3 Mounting the PI cam & HC-SR04

The final task to finish building the SaRQ mechanically was to create a mount for the PI cam and the HC-SR04. The mount was created out of a sheet of Perspex that I drilled a screw on the top face of before tapping it. The tricky thing with plastic is that it melts with heat. This meant that the hole would be drilled and then as soon as I removed the drill bit, the hole filled up with plastic again. After several attempts, I managed to make a hole and screwed the servo motor onto it. The plastic component to hold the HC-SR04 (see Designing the SaRQ for more detail) was put atop the servo to rotate the HC-SR04. To attach the component to the servo motor, a hole was drilled that was the size of the top of the servo motor to sit into. After this was completed, I drilled two holes into the chassis to attach the Perspex to the SaRQ itself. The chassis was of course semi-hollow, made of three walls of plastic, but I still managed to tap a thread into a wall for screws to grip. The last section was the Pi cam. I used self-tapping screws for this attachment. This meant I had to drill two holes and then the screws being inserted carved their own thread into the plastic. With the mount completed, the SaRQ's look was complete (see figure 7.23).

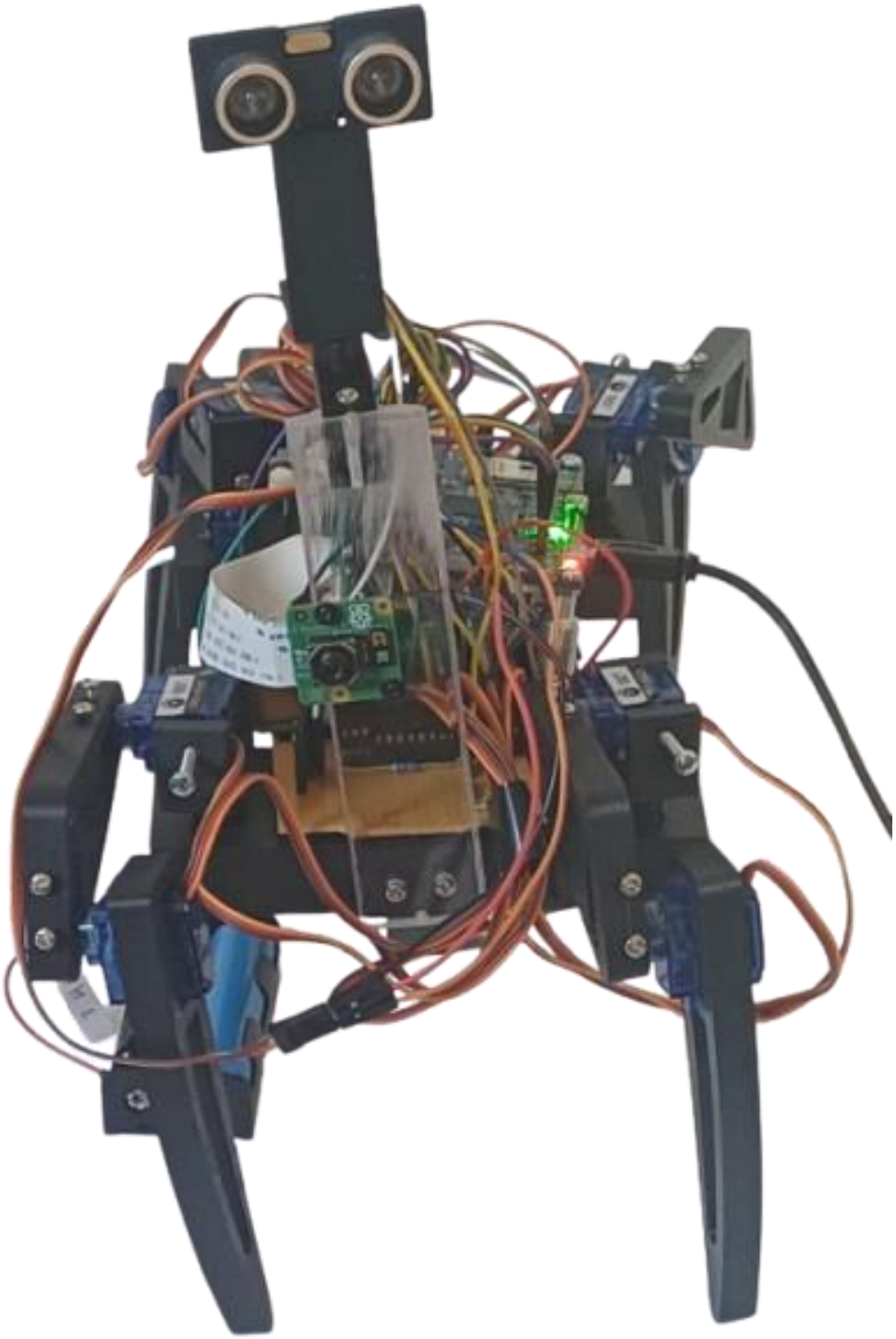


Figure 7.23 The SaRQ with mount included

7.3.4 Integration testing & the SaRQ

The SaRQ underwent a series of testing methods over the development cycle to work cohesively and maintain a solid balance mechanically. The system had to be integrated tightly and when building issues presented themselves that have popped up in the report already such as the connector redesign, testing the servo motors motion to confirm they map to the IK model, stacking the RPI on top of the STM32 to save space and make wiring connections to either side more efficient and adding additional screws to the legs of the build for more robustness. It should be noted the strip board was not in the original scope of the project but due to the nature of the build, the wire management needed to be controlled. This is all relevant integration testing, where new problems presented themselves as everything was coming together but it is not the only examples I have encountered. The actual animation for the SaRQ movement has been omitted until now and for good reason. It was the most difficult portion of the project by far and required several fixes to work in the final project.

7.3.5 Creating a walking animation

Creating the final animation was trial and error as I negotiated between creating an animation that was repeatable and working with the forces of the SaRQ for it to stay stable while taking the largest possible steps. Many approaches were used to find the result and as I tried more methods, I learned ways of improving the code. The walking animation section of this project consisted of testing methods until results were good enough to pursue.

I moved the hips of the SaRQ out as wide as I could, predicting that it would be a stable base to start from before I began to narrow them. I tried starting the shoulders higher off the ground, above the chassis, and having the shoulders lower down. These all had their own pros and cons. Widening the hips were required for balance, but if I made them too wide, there could be no movement as friction made it impossible for the SaRQ to move forward. Too narrow and the SaRQ would collapse. When the shoulders were above being parallel with the chassis, there was more pressure on the legs and once a leg was raised, the SaRQ collapsed. The pressure was worse if the SaRQ's shoulders started below the main chassis.

Knowing the hips had to be extended and the shoulders in line with the chassis, I attempted to create the first walking animation with my learnings. This involved moving each leg along the floor, until each foot was about 20mm away. This was the sweet spot. If I moved further out, the SaRQ would fall. If I moved less, the steps were barely resulting in movement. This is where the inaccuracy of the servo motors was readily apparent. Moving forward, some legs were further ahead with the same input. This was down to the servo motors in some cases, and in others the weight was not evenly distributed. Some legs were pushed down more due to the centre of gravity of the SaRQ, meaning the project would collapse if I raised certain legs that bore the brunt of the weight. This force meant that the legs did not move out as far as there was a higher force pinning them. I adjusted the strip board, RPI and STM32IOT1A but it was still an issue. I tested rubber bands on the legs, hoping to gain a better grip when moving forward, only to see a friction force that was too high for the leg to move. A leg would drag with the rubber band halting it from moving forward and in cases where it did move, the grip was too strong when the leg pulled backwards, meaning the chassis did not follow and the movement stopped. This could not be used for a repeatable animation.

I realised at this point that more screws were needed to for the SaRQ to hold its weight. It collapsed too easily and after adjusting that, the goal was to find the most stable starting position and to work from there. Since widening the hips provided a better base, I made an animation whereby the legs were narrow at the front, but the legs were wide in the rear. Each leg was still dragging along the floor, but some could raise which was an improvement. Weight distribution was a priority to fix but before I could address this the test for this animation was performed.

The legs could enter the position, each leg moving one at a time, but when it came to all four moving together to create motion, the SaRQ did not move forward. The problem was, in my opinion, the animation itself. This method would not generate linear motion as the legs were sweeping outwards and no debugging would fix that.

I tried adjusting the hips once more, this time deciding to take my time to calibrate each servo motor to be the same distance in front of the SaRQ and with the shoulder exactly parallel with

the floor. Before this, there were errors that I felt could be adjusted later but I decided to face them head on. Each leg started at a position slightly differently given the same inputs which made balance more difficult. I knew that implementing an offset for the initial standing animation would be worth an attempt. I chose the front left leg as the one to model the other legs off of. Every other leg had to be the same distance out and the shoulder servo motors all had to line up for the SaRQ chassis to be parallel with the floor. Measuring how parallel something was is relatively simple, looking by eye you could tell if something is in line with the table but in my case, using a ruler for the distances that the top edge of a servo holder was from the ground and how far the tip was out (see figure 7.24)

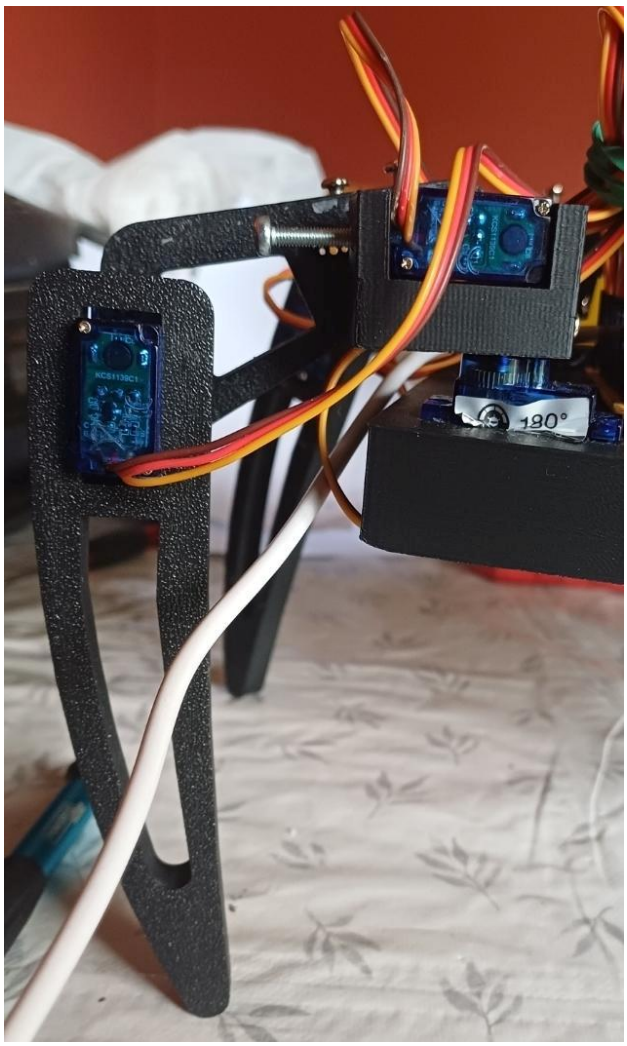


Figure 7.24 A leg in need of offsets to be parallel to the ground

Some shoulders differed by 10 degrees which was a shock. The rear legs had more force exerted on them due to the nature of the centre of gravity of the SaRQ and I made the hip angles wider to provide more stability when walking forward. I rotated the rear legs to see if that would provide a more stable base but it they were more stable in the original position (see figure 7.10 in designing the SaRQ).

The reversed legs were tested by raising legs and seeing how the SaRQ balanced. This was my most common means of testing new methods and by now I could quickly see the potential based on preliminary tests. The reversed legs were much more unstable. The rotated legs did not distribute the weight better than the initial setup and I quickly reverted to the original leg setup. Once I calculated all the offsets, I started the process of stepping forward. This time due to how stable the system was with the better starting point and extra screws; the legs could raise off the ground rather than drag along it. I tested the rubber bands to see if they would provide better grip but unfortunately, they did not, only causing the SaRQ to be stuck when trying to step forward, the front legs caught gripping the ground but the servo motors not strong enough to drag the entire system forward. I removed the rubber bands after this test.

I had to raise the legs in a particular order for the sake of balance, learning that the rear left leg was the most unstable and had to go first, the front right next as raising a diagonal was more stable than moving the leg opposite. The other rear moved forward, followed by the front left to complete the setup. Every leg pulled forward in unison and a step occurred. This was much better than the original step forward animation for a multitude of reasons. It was more stable and repeatable, the legs actually walked, stepping off the ground and not dragging along it. Due to raising off of the ground, the step meant a bigger gain forward as the first step animation meant the SaRQ was dragging itself and friction reduced the gain forward. This was my first success, but due to the process of getting to this point I had spent countless hours testing means of making the SaRQ more stable and finding the correct animation to use. It was worth it once I saw the project step forward and I am quite happy with the result.

8 Project Plan

Over the course of this project, I used agile methodologies to plan sprints in Jira to complete this project. I attended weekly standups where I informed my peers of my progress from the last week and what I intended to complete that week and whether I had blockers. I would keep in close contact with my supervisor, updating him on my progress and asking for his input on my thought process regarding how I planned to attack problems that arose. The planned timeline for pre-Christmas was followed closely in reality with the only deviation being the inverse kinematics not being fully complete pre-Christmas. Note I completed it during the Christmas break before returning to college to keep on track.

Timeline pre-Christmas (October 6th until December 21st):

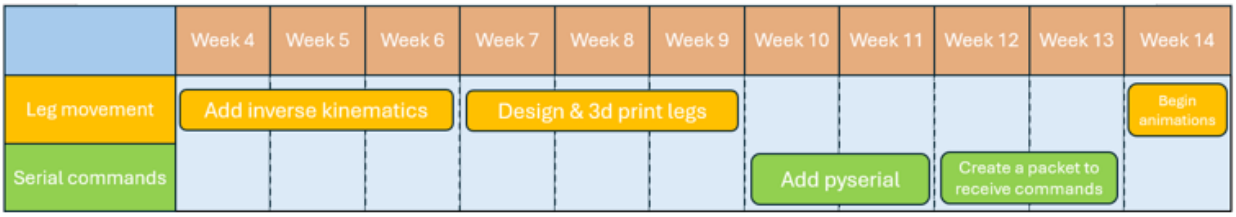


Figure 8.1 The planned timeline leading up to Christmas

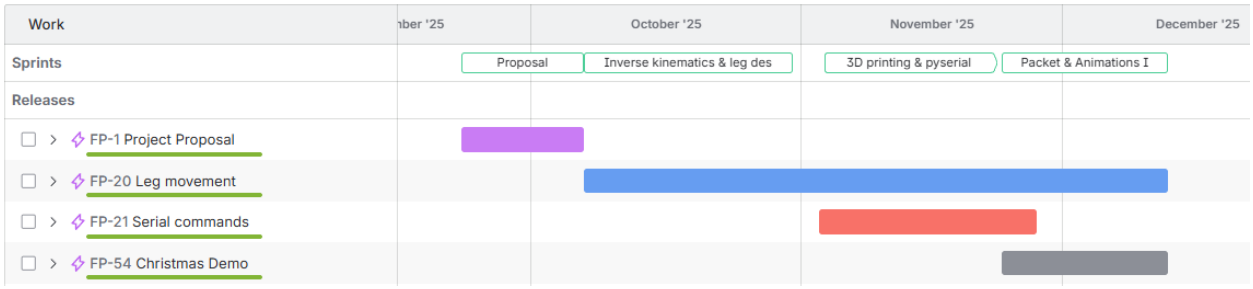


Figure 8.2 The timeline pre-Christmas in Jira

In terms of post-Christmas, I started completing tasks ahead of schedule, managing to add YOLO11 and implementing an SMTP server ahead of schedule. In there is one issue, however. This is the animations epic. I had to work on walking for the entire second half of the timeline due to the range of issues I encountered. This is the only deviation on an otherwise successfully planned and executed plan.

Timeline post-Christmas (Jan 19th until May 1st):

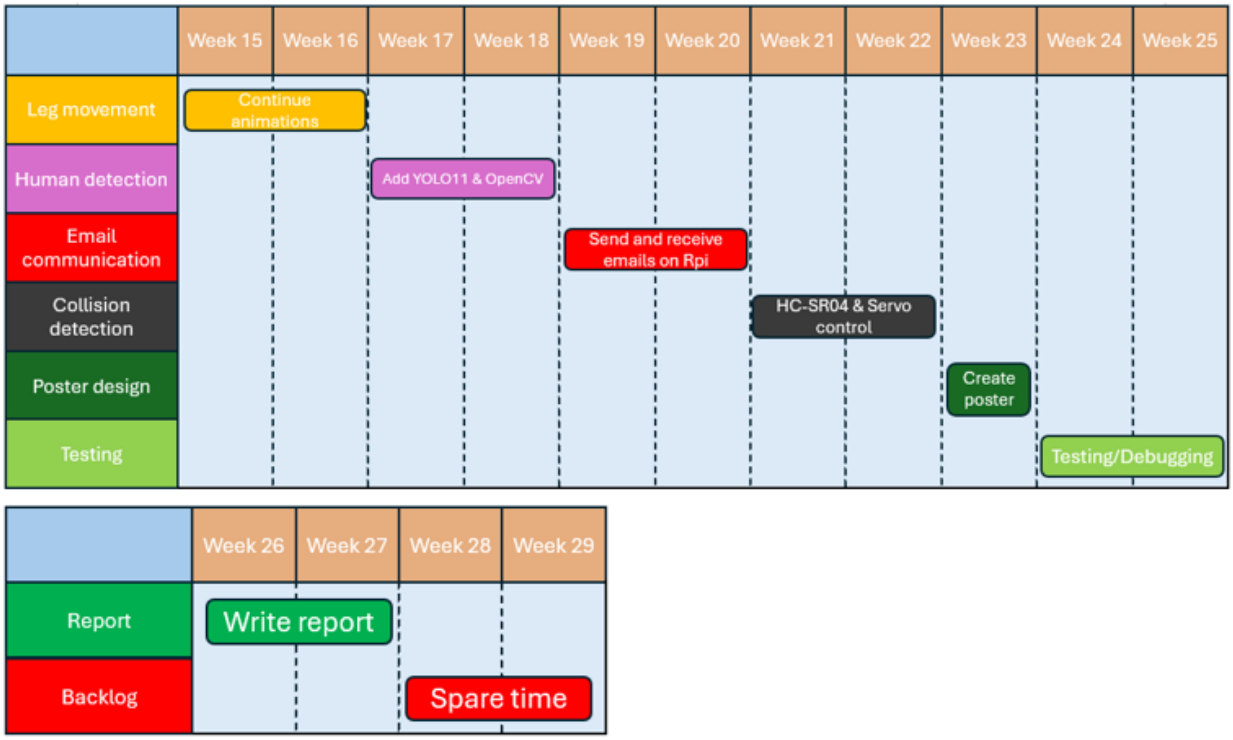


Figure 8.3 The timeline post-Christmas

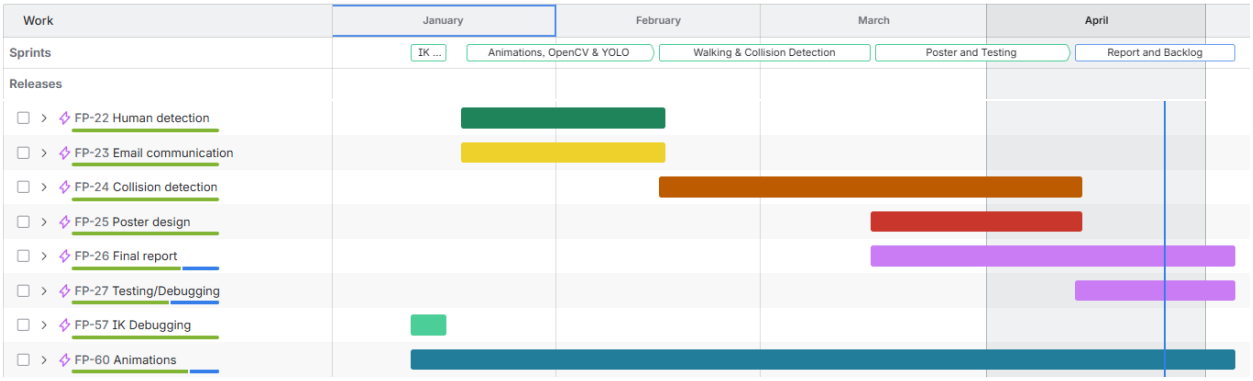


Figure 8.4 The timeline post-Christmas in Jira

9 Ethics

This project set out to solve an ethical dilemma, do you send someone into an unstable building, to risk their life on the chance that someone may be in there? The question aside, there are considerations to be taken when speaking on the project.

The design of the SaRQ's legs and chassis are plastic and not environmentally sustainable. The hope would be to change the material to something less harmful to the environment at a later point in time but that is not the only ethical point to consider.

The SaRQ not only takes images of humans, but without their consent and in times of great distress. The ethical dilemma here is that the SaRQ may be causing distress to people in searching for them and taking their photos, but with the result being that their chance at being aided is increased. The SaRQ would dispose of any images after a disaster has taken place, and it should also be noted that images can be taken in public places [46]. Security measures should be put in place so that only certain individuals should have access to the images taken and it should be made certain that they are responsible for the disposal of the images.

A final note should be the fact that YOLO is a machine learning algorithm, and many such algorithms have environmental issues associated with them. Machine learning can impact energy and water resources and while in some cases, they aid sustainability, in the case of this project it is performed for search and rescue [47].

10 Conclusion

I believe this project met the goals set out before development, and while there are improvements I would make in the future, I know that my understanding and problem solving have improved immensely thanks to this project. I have gained experience in machine learning optimisation and implementation, first-hand knowledge implementing serial communication and research of multiple protocols, understanding of theory of operation and programming of devices such as the RPI itself, the HC-SR04, Pi cam along with servers such as the SMTP and MJPEG server running on the RPI. I would say that integrating all of these pieces also taught me how they interact with each other and work in a cohesive system.

The SaRQ has been built to be sturdy and rigid, with a mount, chassis, legs and holder for the ultrasonic all designed and assembled over time. I have created a walking animation for the SaRQ to move forward which took the longest portion of this project, but I am the proudest of this achievement. I integrated several systems such as UART for communication, YOLOv11 for human detection, SMTP for alerting SARP of a human, a HC-SR04 for collision detection and servo programming with PWM.

The project has a working prototype where the features work together, but if I had to develop it further there is one thing I would focus on. I would make the walking more robust, potentially making a hexapod for better stability. This would require more servo motors and pins on the STM32IOT1A, but I believe the walking would be much smoother as a result.

11 References

[1] alldatasheet.com, “MG90S Datasheet(PDF) - List of Unclassified Manufacturers,” Accessed Oct. 27th, 2025. [Online]. Available: *www.alldatasheet.com*. https://www.alldatasheet.com/datasheet-pdf/pdf/1132104/ETC2/MG90S.html

[2]“Adeept AD002 Servo Motor for RC Robot Car, Compatible with MG90S Sevo, Metal Gear,” Accessed Oct. 27th, 2025. [Online]. Available: *www.adeept.com*. https://www.adeept.com/ad002-servo-motorx2_p0274.html

[3] “Ultra-low-power Arm® Cortex®-M4 32-bit MCU+FPU, 100DMIPS, up to 1MB Flash, 128 KB SRAM, USB OTG FS, analog, audio” St.com. 2019. Accessed Oct. 22nd, 2025. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32l475vg.pdf

[4] “UM2153 User manual Discovery kit for IoT node, multi-channel communication with STM32L4,” St.com. 2019. Accessed Oct. 22nd, 2025. [Online]. Available: https://www.st.com/resource/en/user_manual/um2153-discovery-kit-for-iot-node-multichannel-communication-with-stm32l4-stmicroelectronics.pdf

[5] S. Campbell, “Basics of the I2C Communication Protocol,” *Circuit Basics*, Feb. 13, 2016. Accessed Apr. 2nd, 2026. [Online]. https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/

[6] P. Dhaker, “Introduction to SPI Interface | Analog Devices,” *Analog.com*, Sep. 2018. Accessed Apr. 2nd, 2026. [Online]. https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html

[7] E. Pena and M. G. Legaspi, “UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter | Analog Devices,” *Analog.com*, 2024. Accessed Apr. 2nd, 2026. [Online]. https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html

[8] “Introduction Description of STM32L4/L4+ HAL and low-layer drivers UM1884 User manual.” Accessed Oct. 22nd, 2025.
[Online]. Available: https://www.st.com/resource/en/user_manual/um1884-description-of-stm32l4l4-hal-and-lowlayer-drivers-stmicroelectronics.pdf

[9] Ampheo, “UART Serial Communication Experiment Based on Raspberry Pi 4B and STM32,” *Medium*, Apr. 14, 2025. Accessed Oct. 22nd, 2025.
[Online]. Available: https://medium.com/@pqshedy33/uart-serial-communication-experiment-based-on-raspberry-pi-4b-and-stm32-5abe5fe8f152

[10] “time — Time access and conversions — Python 3.10.5 documentation,” *docs.python.org*. Accessed Apr. 2nd, 2026.
[Online]. Available: https://docs.python.org/3/library/time.html#time.time

[11] “pySerial API — pySerial 3.4 documentation,” *pyserial.readthedocs.io*. Accessed Apr. 2nd, 2026. [Online]. Available: https://pyserial.readthedocs.io/en/latest/pyserial_api.html

[12] “Raspberry Pi 4 Model B,” 2026. Accessed Apr. 20th, 2026. [Online] Available: https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008344-DS-5-raspberry-pi-4-product-brief.pdf?disposition=inline

[13] “Raspberry Pi Documentation - Processors,” *www.raspberrypi.com*. Accessed Apr. 20th, 2026. [Online] Available:
https://www.raspberrypi.com/documentation/computers/processors.html

[14] “BCM2711 ARM peripherals” 2022. Accessed Apr. 10th, 2026. [Online] Available:
https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008248-DS-1-bcm2711-peripherals.pdf?disposition=inline

[15] JustAnotherMakerChannel, “Robot Inverse Kinematics With A Hexapod Leg,” *YouTube*, May 24, 2023. Accessed Oct. 1, 2025. [Online]
Available: https://www.youtube.com/watch?v=HjmIOKSp7v4

[16] MathWorks, “What Is Inverse Kinematics?,” Accessed Apr. 2nd, 2026. [Online]. Available: *www.mathworks.com*. https://www.mathworks.com/discovery/inverse-kinematics.html

[17] JustAnotherMakerChannel, “InverseKinematicsExample/Inverse_Kinematics_WritingLines.ino at main · JustAnotherMakerChannel/InverseKinematicsExample,” *GitHub*, 2025. Accessed Jan. 10th, 2026. [Online]. Available: https://github.com/JustAnotherMakerChannel/InverseKinematicsExample/blob/main/Inverse_Kinematics_WritingLines.ino

[18] Chen Hao, “MOST COMPLETE Inverse Kinematics Guide for Hexapod Servo Calibration - IK - Coding - Testing,” *YouTube*, Jul. 26, 2025. Accessed Jan. 11th, 2026. [Online]. Available: https://www.youtube.com/watch?v=Q1kFkpIGSQg

[19] “inverse kinematics 2,” *Desmos*, 2026. Accessed Jan. 12th, 2026. [Online]. Available: https://www.desmos.com/calculator/2uzkupqa2t

[20] Amit Kharche, “Object Detection Models Explained: R-CNN, YOLO, SSD,” *Medium*, Jul. 28, 2025. Accessed Jan. 25th, 2026. [Online]. Available: https://medium.com/%40amitkharche/object-detection-models-explained-r-cnn-yolo-ssd-49b607c9ef6d

[21] GeeksforGeeks, “Convolutional Neural Network (CNN) in Machine Learning,” *GeeksforGeeks*, Dec. 25, 2020. Accessed Apr. 3rd, 2026. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/

[22] Z. Keita, “YOLO Object Detection Explained,” *Datacamp.com*, Sep. 28, 2022. Accessed Jan. 25th, 2026. [Online]. https://www.datacamp.com/blog/yolo-object-detection-explained?dc_referrer=https%3A%2F%2Feuc-onenote.officeapps.live.com%2F

[23] “Max Pooling in CNNs: Why It Matters and How It Works,” *Medium*, May 16, 2025. Accessed Apr. 3rd, 2026. [Online]. Available: http://medium.com/@onally.sourour9/max-pooling-in-cnns-why-it-matters-and-how-it-works-60c590bca8b9

[24] GeeksforGeeks, “What are Convolution Layers?,” *GeeksforGeeks*, Jun. 11, 2024. Accessed Apr. 3rd, 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/what-are-convolution-layers/

[25] GeeksforGeeks, “What is Fully Connected Layer in Deep Learning?,” *GeeksforGeeks*, May 27, 2024. Accessed Apr. 3rd, 2026. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/what-is-fully-connected-layer-in-deep-learning/

[26] D. Dominguez, “Understanding Inference-Time Compute - Daniel Dominguez - Medium,” *Medium*, Jan. 27, 2025. Accessed Apr. 3rd, 2026. [Online]. Available: https://dominguezdaniel.medium.com/understanding-inference-time-compute-c6c6ca2e17a8

[27] “Getting Started with YOLO Object and Animal Recognition on the Raspberry Pi - Tutorial Australia,” *Core Electronics*, Sep. 27, 2024. Accessed Jan. 20th, 2026. [Online]. Available: https://core-electronics.com.au/guides/raspberry-pi/getting-started-with-yolo-object-and-animal-recognition-on-the-raspberry-pi/

[28] NimaSalehiM, “Issue with Installation on Python 3.13,” . Accessed Feb. 8th, 2026. [Online]. Available: *GitHub*, Dec. 08, 2024. https://github.com/ultralytics/ultralytics/issues/18114

[29] “Raspberry Pi: Send Email using Python (SMTP Server) | Random Nerd Tutorials,” *Random Nerd Tutorials*, Jul. 12, 2023. Accessed Oct. 1, 2025. [Online]
Available: https://randomnerdtutorials.com/raspberry-pi-send-email-python-smtp-server/

[30] Guvv, “How to attach a file into a email with python?,” *Stack Overflow*, Apr. 29, 2021. Accessed Feb. 3, 2026. [Online]
Available: https://stackoverflow.com/questions/67318514/how-to-attach-a-file-into-a-email-with-python

[31] “Mike Kohn!,” *Mikekohn.net*, 2024. Accessed Apr. 7th, 2026. [Online]. Available: https://www.mikekohn.net/software/mjpeg_webserver.php

[32] R. Santos, “Raspberry Pi: MJPEG Streaming Web Server (Picamera2) | Random Nerd Tutorials,” *Random Nerd Tutorials*, May 09, 2024. Accessed Apr. 7th, 2026. [Online]. Available: https://randomnerdtutorials.com/raspberry-pi-mjpeg-streaming-web-server-picamera2/

[33] “threading — Thread-based parallelism,” *Python documentation*, 2026. Accessed Apr. 10th, 2026. [Online]. Available: https://docs.python.org/3/library/threading.html#condition-objects

[34] “socketserver — A framework for network servers,” *Python documentation*. Accessed Apr. 10th, 2026. [Online]. Available: https://docs.python.org/3/library/socketserver.html

[35] “http.server — HTTP servers,” *Python documentation*, 2026. Accessed Apr. 10th, 2026. [Online]. Available: https://docs.python.org/3/library/http.server.html#

[36] “The Picamera2 Library A libcamera-based Python library for Raspberry Pi cameras.” Accessed Apr. 10th, 2026. [Online]. Available: https://pip-assets.raspberrypi.com/categories/652-raspberry-pi-camera-module-2/documents/RP-008156-DS-2-picamera2-manual.pdf?disposition=inline

[37] “301 Moved Permanently - HTTP | MDN,” *MDN Web Docs*, Mar. 13, 2025. Accessed Apr. 10th, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/301

[38] N. Freed and N. Borenstein, “Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types,” *www.rfc-editor.org*, Nov. 1996, Accessed Apr. 10th, 2026. [Online]. Available: https://doi.org/10.17487/RFC2046.

[39] “2D vs 3D LiDAR : comparison of two Laser Scanning Technologies - YellowScan,” *YellowScan*, 2022. Accessed Apr. 8, 2026. [Online] Available: https://www.yellowscan.com/knowledge/2d-vs-3d-lidar-a-comparison-of-two-laser-scanning-technologies/

[40] A. Prabhu, “How HC-SR04 Ultrasonic Sensor Works & How to Interface It With Arduino,” *Last Minute Engineers*, Jul. 03, 2018. Accessed Apr. 8, 2026. [Online] Available: https://lastminuteengineers.com/arduino-sr04-ultrasonic-sensor-tutorial/#

[41] C. Garrett, “Previously we looked at motor control but for our robot there is another type of actuator that we…,” *Medium*, Apr. 06, 2018. Accessed Feb. 25th, 2026. [Online] Available: https://medium.com/@makerhacks/jitter-free-servo-control-on-the-raspberry-pi-f596bf07d09f

[42] B. Croston, “RPi.GPIO: A module to control Raspberry Pi GPIO channels,” *PyPI*, Feb. 06, 2022. Accessed Apr. 8, 2026. [Online] Available: https://pypi.org/project/RPi.GPIO/

[43] “Raspberry Pi Tutorial on Connecting the Ultrasonic sensor HC SR04,” *www.youtube.com*. Accessed Feb. 25th, 2026. [Online] Available: https://www.youtube.com/watch?v=_7drIUmC8Zo

[44] Wikipedia, “Speed of Sound,” *Wikipedia*, May 20, 2019. Accessed

Apr. 8th, 2026. [Online] Available: https://en.wikipedia.org/wiki/Speed_of_sound

[45] element14 presents, “How to Build a Quadruped Robot - NO MATH!,” *YouTube*, Feb. 18, 2022. Accessed Oct. 28th, 2026. [Online]. Available: https://www.youtube.com/watch?v=BT6EySoEQPU

[46] “What is the position regarding individuals taking photographs/videos in a public place? | Data Protection Commission,” *What is the position regarding individuals taking photographs/videos in a public place? | Data Protection Commission*, 2025. Accessed Apr. 10th, 2026. [Online] Available: https://www.dataprotection.ie/en/faqs/topical-data-protection-issues/what-position-regarding-individuals-taking-photographsvideos-public-place

[47] M. Sloan, “Climate change and machine learning — the good, bad, and unknown | MIT Sloan,” *MIT Sloan*, Feb. 10, 2025. Accessed Apr. 10th, 2026. [Online] Available: https://mitsloan.mit.edu/ideas-made-to-matter/climate-change-and-machine-learning-good-bad-and-unknown